FRE7241 Algorithmic Portfolio Management

Lecture#7, Spring 2026

Jerzy Pawlowski *jp3900@nyu.edu*

NYU Tandon School of Engineering

March 10, 2026

NYU | TANDON SCHOOL OF ENGINEERING

Single Period Binary Gamble

Consider a single investment (gamble) with a binary outcome:
The investor makes no up-front payments, and either wins an amount a (with probability p), or loses an amount b (with probability $q = 1 - p$).

	win	lose
probability	p	q = 1 - p
payout	a	-b
terminal wealth	1 + a	1 - b

The initial wealth is equal to 1 dollar, and the terminal wealth after the gamble is either $1 + a$ (with probability p), or $1 - b$ (with probability $q = 1 - p$).

The amounts a and b are expressed as percentages of the wealth risked in the gamble, and the ratio a/b is called the *betting odds*.

The expected return on the gamble is called the *edge* and is equal to: $\mu = p\,a - q\,b$, and the variance of returns is equal to: $\sigma^2 = p\,q\,(a+b)^2$.

If the investor chooses to risk only a fraction k_r of wealth, then the return on the gamble is either $k_r a$ (with probability p), or $-k_r b$ (with probability $q = 1 - p$).

The fraction k_r can be greater than 1 (leveraged investing), or it can be negative (shorting).

And the expected return on the gamble is equal to: $p\,k_r a - q\,k_r b = k_r\,\mu$.

If an investor makes decisions exclusively based on the expected return μ, then they would either invest all their wealth ($k_r = 1$) on the gamble if $\mu > 0$, or choose not to invest at all ($k_r = 0$) if $\mu < 0$.

Without loss of generality we can assume that $p = q = \frac{1}{2}$.

And then $\mu = 0.5\,(a - b)$, and $\sigma^2 = 0.25\,(a+b)^2$.

The *Sharpe ratio* of the gamble is then equal to:

$$S_r = \frac{\mu}{\sigma} = \frac{(a-b)}{(a+b)}$$

Investor Utility and Fractional Betting

The *expected utility* hypothesis states that investors try to maximize the expected *utility* of wealth, not the expected wealth.

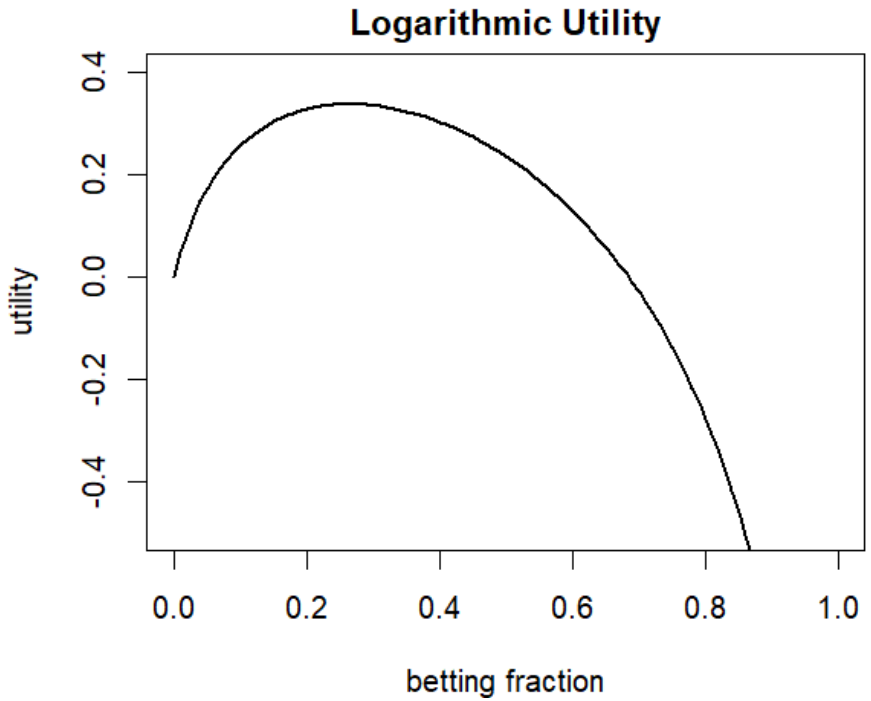
The *logarithmic utility* function is defined as the logarithm of wealth: $u(w) = \log(w)$.

Under *logarithmic utility* investor preferences depend on the percentage change of wealth, instead of the absolute change of wealth: $\mathrm{d}u(w) = \frac{\mathrm{d}w}{w}$.

An investor with *logarithmic utility* invests only a fraction k_r of their wealth in a gamble, depending on the risk-return of the gamble.

If the initial wealth is equal to 1, then the expected value of *logarithmic utility* for the binary gamble is equal to: $u(k_r) = p\ \log(1 + k_r a) + q\ \log(1 - k_r b)$.

In 1738 Daniel Bernoulli introduced the concept of *logarithmic utility* in his work *"Specimen Theoriae Novae de Mensura Sortis"* (New Theory of the Measurement of Risk).




```
> # Define logarithmic utility
> utilfun <- function(frac, p=0.3, a=20, b=1) {
+   p*log(1+frac*a) + (1-p)*log(1-frac*b)
+ }  ##  end utilfun
> # Plot utility
> curve(expr=utilfun, xlim=c(0, 1),
+ ylim=c(-0.5, 0.4), xlab="betting fraction",
+ ylab="utility", main="", lwd=2)
> title(main="Logarithmic Utility", line=0.5)
```

Optimal Fractional Betting Under Logarithmic Utility

The betting fraction that maximizes the *utility* can be found by equating the derivative of *utility* to zero:

$$\frac{\mathrm{d}u(k_r)}{\mathrm{d}k_r} = \frac{p\,a}{1+k_r a} - \frac{q\,b}{1-k_r b} = 0$$

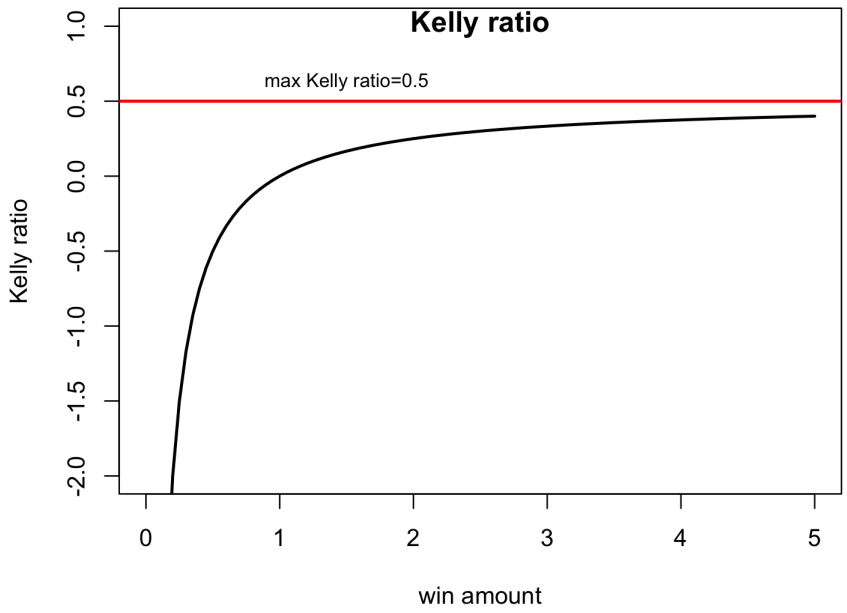
$$k_r = \frac{p}{b} - \frac{q}{a} = \frac{p\,a - q\,b}{b\,a} = \frac{\mu}{b\,a}$$

The optimal betting fraction k_r is called the *Kelly ratio*, and it depends on the parameters of the gamble.

The *Kelly ratio* can be greater than 1 (leveraged investing), or it can be negative (shorting).

The *Kelly ratio* is zero if the ratio of the probabilities of losing and winning is equal to the betting odds: $\frac{q}{p} = \frac{a}{b}$.

As the betting odds $\frac{a}{b}$ increase the *Kelly ratio* approaches its maximum value of $\frac{p}{b}$.




```
> # Define and plot Kelly ratio
> kelly_ratio <- function(a, b, p) {
+   p/b - (1-p)/a
+ }  ##  end kelly_ratio
> # Define and plot Kelly ratio
> curve(expr=kelly_ratio(x, p=0.5, b=1), xlim=c(0, 5),
+ ylim=c(-2, 1), xlab="win amount",
+ ylab="Kelly ratio", main="", lwd=2)
> abline(h=0.5, lwd=2, col="red")
> text(x=1.5, y=0.5, pos=3, cex=0.8, labels="max Kelly ratio=0.5")
> title(main="Kelly ratio", line=-0.8)
```

The Kelly Criterion

The *Kelly criterion* states that investors who want to maximize their logarithmic utility should bet a fraction of their capital equal to the *Kelly ratio*.

Investors with concave utility functions (for example logarithmic utility) are sensitive to the risk of ruin (losing all their capital).

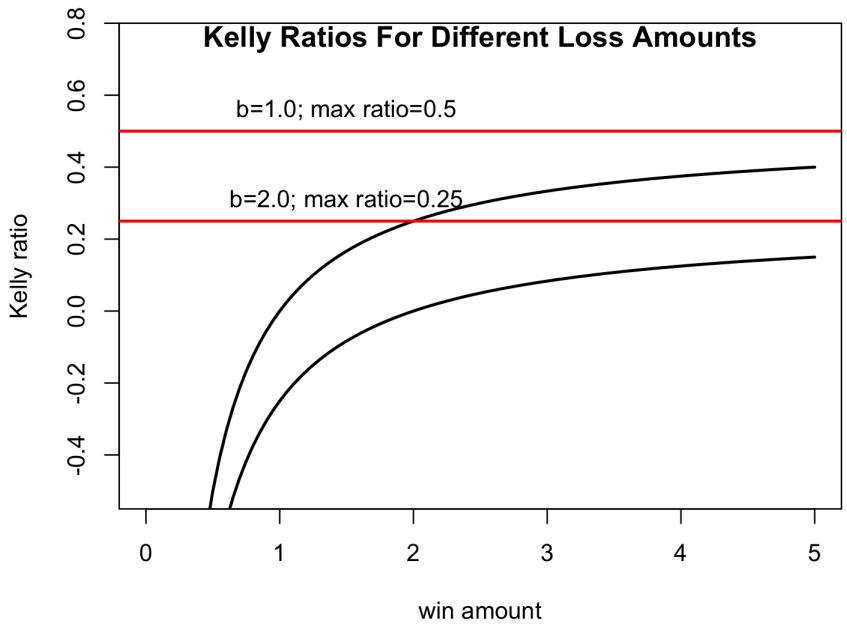
Applying the *Kelly criterion* and betting only a fraction of their capital reduces the risk of ruin (but it doesn't eliminate the risk if prices drop suddenly).

The loss amount b determines the risk of ruin, with larger values of b increasing the risk of ruin.

Therefore investors will choose a smaller betting fraction k_r for larger values of b.

This means that even for huge odds in their favor, investors may not choose to invest all their capital, because of the risk of ruin.

For example, if the betting odds are very large $a \to \infty$, then the *Kelly ratio*: $k_r = \frac{p}{b}$.




```
> # Plot several Kelly curves
> curve(expr=kelly_ratio(x, p=0.5, b=1), xlim=c(0, 5),
+ ylim=c(-0.5, 0.75), xlab="win amount",
+ ylab="Kelly ratio", main="", lwd=2)
> abline(h=0.5, lwd=2, col="red")
> text(x=1.5, y=0.5, pos=3, cex=1.0, labels="b=1.0; max ratio=0.5")
> curve(expr=kelly_ratio(x, p=0.5, b=2), add=TRUE, main="", lwd=2)
> abline(h=0.25, lwd=2, col="red")
> text(x=1.5, y=0.25, pos=3, cex=1.0, labels="b=2.0; max ratio=0.25"
> title(main="Kelly Ratios For Different Loss Amounts", line=-0.8)
```

Utility of Multiperiod Betting

In multiperiod betting the investor participates in multiple rounds of sucessive gambles, and in each round they risk a fixed fraction k_f of their current outstanding wealth.

Let r_t be the random return on the gamble in period i, and let $w_i = (1 + k_f r_t)$ be the random wealth increment.

Then the terminal wealth after n rounds is equal to the compounded wealth increments:
$w_n = \prod_{i=1}^{n} w_i = \prod_{i=1}^{n}(1 + k_f r_t)$.

And the utility is equal to the sum of the individual utilities:

$$u_n = \log(w_n) = \sum_{i=1}^{n} \log(w_i) = \sum_{i=1}^{n} \log(1+k_f r_t) = \sum_{i=1}^{n} u_i$$

The individual utilities are all maximized by the same fraction $k_f = k_r$ equal to the *Kelly ratio*, so the *Kelly ratio* for multiperiod betting is the same as for single period betting:

$$k_f = \frac{p}{b} - \frac{q}{a} = k_r$$

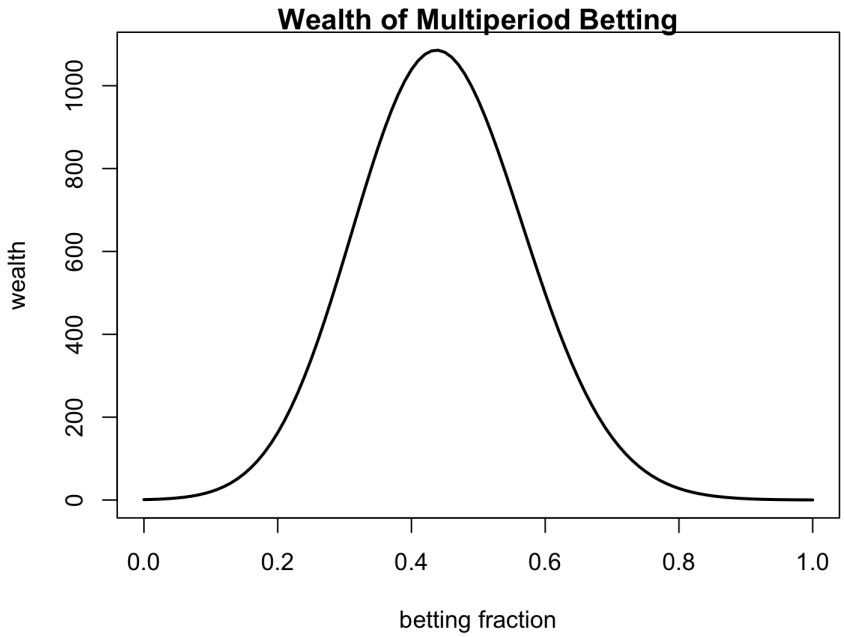
Wealth of Multiperiod Betting

In multiperiod betting the investor participates in n rounds of sucessive gambles, and in each round they risk a fixed fraction k_f of their current outstanding wealth.

In each round the wealth is multiplied by either $(1 + k_f a)$ (win) or $(1 - k_f b)$ (loss), so that the current outstanding wealth changes over time.

The terminal wealth after n rounds with m wins is equal to: $w(k_f) = (1 + k_f a)^m (1 - k_f b)^{n-m}$.

If the number of rounds n is very large, then the number of wins is almost always equal to $m = n\,p$, and the terminal wealth is equal to: $w(k_f) = (1 + k_f a)^{np} (1 - k_f b)^{nq}$.




```
> # Wealth of multiperiod binary betting
> wealthv <- function(f, a, b, n, p) {
+   m <- n*p
+   (1+f*a)^m * (1-f*b)^(n-m)
+ }  ##  end wealth
> curve(expr=wealthv(x, a=0.8, b=0.1, n=1e3, p=0.15), xlim=c(0, 1),
+ xlab="betting fraction",
+ ylab="wealth", main="", lwd=2)
> title(main="Wealth of Multiperiod Betting", line=0.1)
```

Optimal Multiperiod Betting

The betting fraction k_f that maximizes the terminal wealth is found by setting the derivative of $w(k_f)$ to zero:

$$\frac{\mathrm{d}w(k_f)}{\mathrm{d}k_f} = npa(1+k_f a)^{np-1}(1-k_f b)^{nq} - nqb(1+k_f a)^{np}(1-k_f b)^{nq-1}$$

$$= (\frac{npa}{1+k_f a} - \frac{nqb}{1-k_f b})(1+k_f a)^{np}(1-k_f b)^{nq} = 0$$

We can then solve for the optimal betting fraction k_f:

$$\frac{pa}{1+k_f a} - \frac{qb}{1-k_f b} = 0$$

$$pa(1-k_f b) - qb(1+k_f a) = 0$$

$$pa - qb - k_f ab = 0$$

$$k_f = \frac{pa-qb}{ab} = \frac{p}{b} - \frac{q}{a} = k_r$$

The above is just the *Kelly ratio* k_r that maximizes the utility.

So the *Kelly ratio* k_r that maximizes the utility also maximizes the terminal wealth.

Investing With Fixed Margin

Let r_t be the percentage returns on a *risky asset*, so that the stock price p_t at time t is given by:

$$p_t = p_0 \prod_{i=1}^{t} (1 + r_t)$$

The initial investor wealth at time $t = 0$ is equal to 1 dollar, and they also borrow on margin m dollars to invest in the *risky asset*.

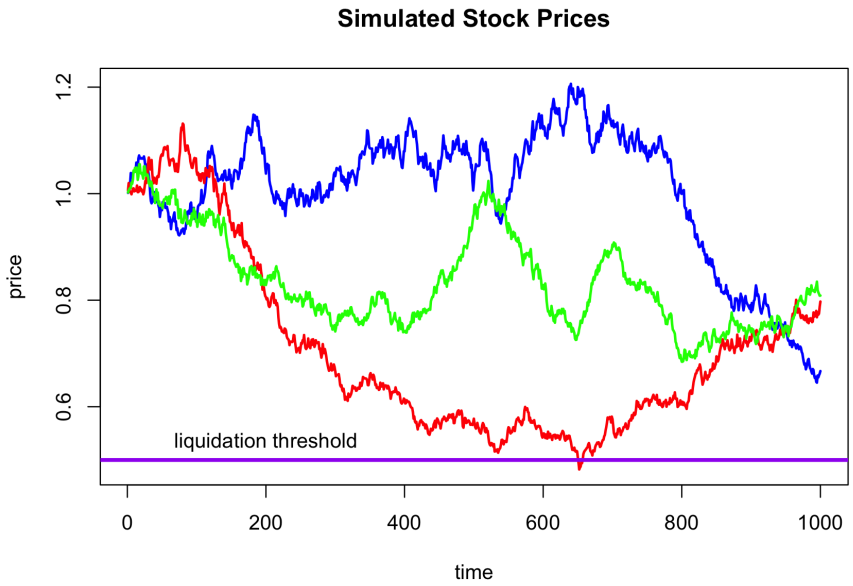
The investor's *wealth* at time t is equal to (the margin borrowing rate is assumed to be zero):

$$w_t = 1 + m \frac{p_t - p_0}{p_0}$$

The *leverage* k_r is equal to the *margin debt* m divided by the total wealth w_t: $k_r = m/w_t$.

If the stock price drops then the *leverage* increases, because the *margin debt* is fixed while the wealth drops.

If the stock price drops enough so that the wealth reaches zero, then the investment is liquidated and the investor is ruined.




```
> matplot(pricep, type="l", lwd=2,
+   main="Simulated Stock Prices",
+   ylim=range(pricep), col=colorv,
+   lty="solid", xlab="time", ylab="price")
> abline(h=0.5, col="purple", lwd=3)
> text(x=200, y=0.5, pos=3, labels="liquidation threshold")
```

Investing With Fixed Leverage

In order to avoid ruin, the investor may choose to maintain a fixed *leverage ratio* equal to k_r, so that the amount invested in the *risky asset* is proportional to the *wealth*: $k_r\, w_t$.

This requires buying the *risky asset* when its price increases, and selling it when it drops.

The return on the *risky asset* in a single period is equal to: $k_r\, w_t\, r_t$, so the *terminal wealth* at time t is equal to the compounded returns:

$$w_t = (1 + k_r\, r_1) \ldots (1 + k_r\, r_t) = \prod_{i=1}^{t} (1 + k_r\, r_t)$$

The utility of the *terminal wealth* is equal to the sum of the utilities of single periods:

$$\mathbb{E}[\log w_t] = \mathbb{E}[\log((1 + k_r\, r_1) \ldots (1 + k_r\, r_t))]$$

$$= \sum_{i=1}^{t} \mathbb{E}[\log(1 + k_r\, r_t)] = t\, \mathbb{E}[\log(1 + k_r\, r)]$$

The last equality holds because all the utilities of single periods are the same.

Let the returns over a short time period be equal to r, with probability distribution $p(r)$.

The mean return $\bar{r}$, and variance σ^2 are:

$$\bar{r} = \int r\, p(r)\, \mathrm{d}r\ ; \quad \sigma^2 = \int (r - \bar{r})^2\, p(r)\, \mathrm{d}r$$

Since the returns are over a short time period, we have: $r \ll 1$ and $\bar{r} \ll \sigma$, so that we can replace $r - \bar{r}$ with r as follows:

$$\int (r - \bar{r})^2\, p(r)\, \mathrm{d}r \approx \int r^2\, p(r)\, \mathrm{d}r$$

Utility of Leveraged Asset Returns

So the utility of the *terminal wealth* u_t is equal to the utility of a single period times the number of periods:

$$u_t = \mathbb{E}[\log w_t] = t\, \mathbb{E}[\log (1 + k_r\, r)] = t\, u_r$$

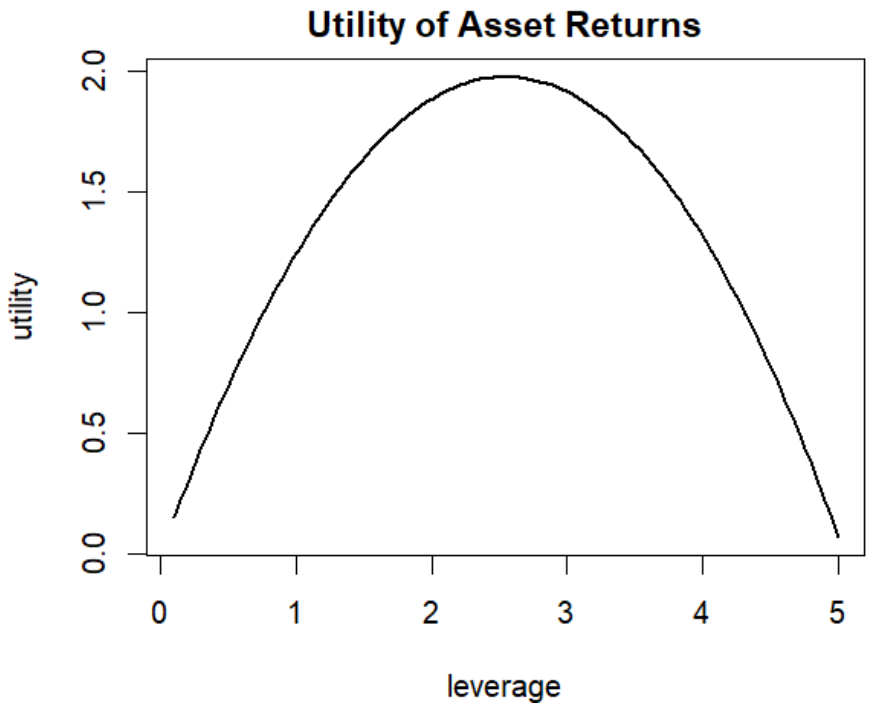
The utility of the asset returns u_r is equal to:

$$u_r = \mathbb{E}[\log (1 + k_r\, r)] = \int \log(1 + k_r\, r)\, p(r)\, \mathrm{d}r$$

The leverage k_r is limited so that $(1 + k_r\, r) > 0$ for all return values r.

If the mean returns are positive, then at first the utility increases with leverage, but only up to a point.

With higher leverage, the negative utility of the time periods with negative returns becomes significant, forcing the aggregate utility to drop.




```
> # Calculate the VTI returns
> retp <- rutils::etfenv$returns$VTI
> retp <- na.omit(retp)
> c(mean=mean(retp), stdev=sd(retp))
> range(retp)
```

```
> # Define vectorized logarithmic utility function
> utilfun <- function(fracv, retp) {
+   sapply(fracv, function(x) sum(log(1 + x*retp)))
+ }  ##  end utilfun
> utilfun(1, retp)
> utilfun(c(1, 4), retp)
> # Plot the logarithmic utility
> curve(expr=utilfun(x, retp=retp),
+ xlim=c(0.1, 5), xlab="leverage", ylab="utility",
+ main="Utility of Asset Returns", lwd=2)
```

Kelly Criterion for Optimal Leverage of Asset Returns

The *logarithmic utility* u_r can be expanded in the moments of the return distribution:

$$u_r = \mathbb{E}[\log(1 + k_r\, r)] = \int \log(1 + k_r\, r)\, p(r)\, \mathrm{d}r$$

$$= \int (k_r\, r - \frac{(k_r\, r)^2}{2} + \frac{(k_r\, r)^3}{3} - \frac{(k_r\, r)^4}{4})\, p(r)\, \mathrm{d}r$$

$$= k_r \bar{r} - \frac{k_r^2 \sigma^2}{2} + \frac{k_r^3 \sigma^3 \varsigma}{3} - \frac{k_r^4 \sigma^4 \kappa}{4}$$

Where $\varsigma = \int \frac{r^3}{\sigma^3}\, p(r)\, \mathrm{d}r$ is the *skewness*, and $\kappa = \int \frac{r^4}{\sigma^4}\, p(r)\, \mathrm{d}r$ is the *kurtosis*.

The *Kelly leverage* which maximizes the *utility* is found by equating the derivative of *utility* to zero:

$$\frac{\mathrm{d}u_r}{\mathrm{d}k_r} = \bar{r} - k_r \sigma^2 + k_r^2 \sigma^3 \varsigma - k_r^3 \sigma^4 \kappa = 0$$

This shows that the logarithmic utility has positive odd derivatives and negative even derivatives.

Assuming that the third and fourth moments $\sigma^4 \varsigma$ and $\sigma^4 \kappa$ are small and can be neglected, we get:

$$k_r = \frac{\bar{r}}{\sigma^2} = \frac{S_r}{\sigma}\ ; \quad u_r = \frac{1}{2}\frac{\bar{r}^2}{\sigma^2} = \frac{1}{2} S_r^2$$

The *Kelly leverage* is *approximately* equal to the *Sharpe ratio* divided by the *standard deviation*.

The optimal utility u_r is *approximately* equal to half the *Sharpe ratio* S_r squared.

The *standard deviation* and *Sharpe ratio* are calculated over the same time interval as the returns (not annualized).

```
> # Approximate Kelly ratio
> mean(retp)/var(retp)
> PerformanceAnalytics::KellyRatio(R=retp, method="full")
> # Kelly leverage
> unlist(optimize(
+   f=function(x) -utilfun(x, retp),
+   interval=c(1, 4)))
```

Wealth Paths for Different Leverage Ratios

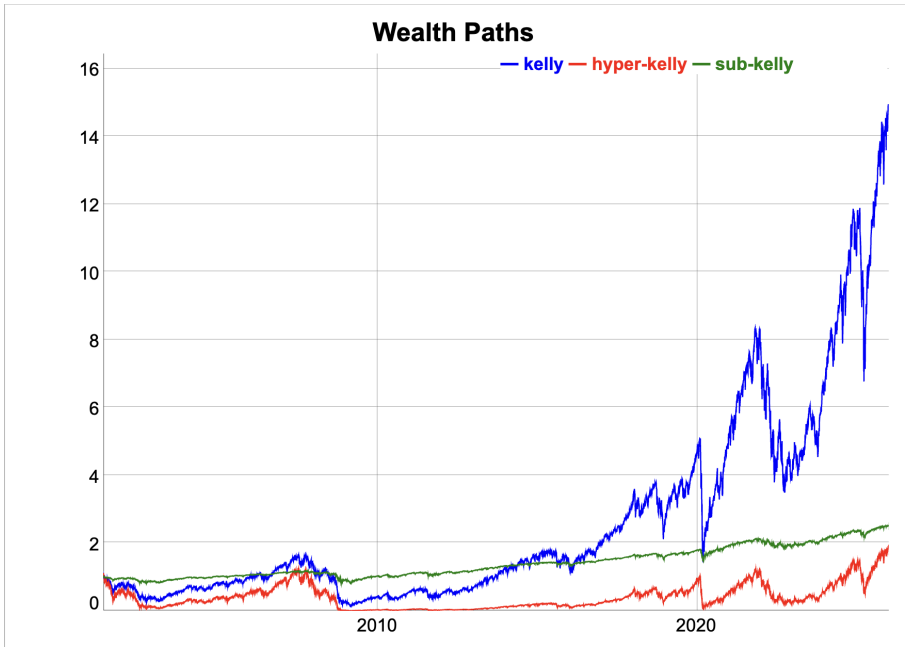
The wealth of a Kelly Strategy with a fixed leverage ratio k_r is equal to:

$$w_t = \prod_{i=1}^{t} (1 + k_r \, r_t)$$

The *Kelly ratio* k_r provides the optimal leverage to maximize the utility of wealth, by balancing the benefit of leveraging higher positive returns, with the risk of ruin due to excessive leverage.

If the mean asset returns are positive, then a higher leverage ratio provides higher returns.

But if the leverage is too high, then the losses in periods with negative returns wipe out most of the wealth, so then it's slow to recover.




```
> # Calculate the VTI returns
> retp <- rutils::etfenv$returns$VTI
> retp <- na.omit(retp)
> # Calculate the wealth paths
> kellyr <- drop(mean(retp)/var(retp))
> wealthk <- cumprod(1 + kellyr*retp)
> wealthyper <- cumprod(1 + (kellyr+2)*retp)
> wealthsub <- cumprod(1 + (kellyr-2)*retp)
> wealthp <- cbind(wealthk, wealthyper, wealthsub)
> colnames(wealthp) <- c("kelly", "hyper-kelly", "sub-kelly")
```

```
> # Dygraph plot o wealth paths
> dygraphs::dygraph(wealthp, main="Wealth Paths") %>%
+   dyOptions(colors=c("blue", "red", "green"), strokeWidth=1) %>%
+   dyLegend(show="always", width=300)
```

Kelly Strategy With Margin Account

The *margin debt* m_t is equal to the dollar amount borrowed to purchase the *risky asset*.

The wealth w_t at time t is equal to the initial wealth $w_0 = 1$ plus the dollar amount of the *risky asset* a_t, minus the *margin debt* m_t: $w_t = 1 + a_t - m_t$.

The dollar amount of the *risky asset* a_t is equal to the wealth w_t times the *leverage* k_r: $a_t = k_r w_t$.

So the *margin debt* m_t is proportional to the wealth w_t: $m_t = (k_r - 1)w_t + 1$.

The wealth changes from w_{t-1} to: $w_t = w_{t-1}(1 + k_r\, r_t)$, while the dollar amount of the *risky asset* changes from $a_{t-1} = k_r w_{t-1}$ to: $a_t = k_r w_{t-1}(1 + r_t)$, so that the leverage changes from k_r to:

$$\frac{k_r w_{t-1}(1 + r_t)}{w_{t-1}(1 + k_r\, r_t)} = \frac{k_r(1 + r_t)}{1 + k_r\, r_t}$$

In order to maintain a fixed *leverage ratio* equal to k_r, the investor must actively trade the *risky asset*, and the *margin debt* m_t changes over time.

The change in margin in a single time period is equal to:

$$\Delta m_t = (k_r - 1)\Delta w_t = k_r(k_r - 1)w_{t-1} r_t$$

The dollar amount of the *risky asset* traded is equal to the change in *margin*.

Therefore the investor must borrow on margin and buy the *risky asset* when its price increases, and sell it when it drops.

Kelly Strategy With Transaction Costs of Trading

The *bid-ask spread* is the percentage difference between the *offer* minus the *bid* price, divided by the *mid* price.

The *bid-ask spread* for liquid stocks can be assumed to be about 10 basis points (bps).

The *transaction costs* c^r due to the *bid-ask spread* are equal to half the *bid-ask spread* δ times the absolute value of the traded dollar amount of the *risky asset*:

$$c^r = \frac{\delta}{2} |\Delta m_t|$$

If the transaction costs are much less than the change in wealth $c^r \ll |\Delta w_t|$, then we can write approximately:

$$c^r = \frac{\delta}{2} k_r (k_r - 1) w_{t-1} |r_t|$$

The wealth of the Kelly Strategy after accounting for the *bid-ask spread* is then equal to:

$$w_t = \prod_{i=1}^{t} (1 + k_r r_t - \frac{\delta}{2} k_r (k_r - 1) |r_t|)$$

The effect of the *bid-ask spread* is to reduce the effective asset returns by an amount proportional to the *bid-ask spread*.

```
> # bidask equal to 1 bp for liquid ETFs
> bidask <- 0.001
> # Calculate the wealth paths
> kellyr <- drop(mean(retp)/var(retp))
> wealthv <- cumprod(1 + kellyr*retp)
> wealth_trans <- cumprod(1 + kellyr*retp -
+   0.5*bidask*kellyr*(kellyr-1)*abs(retp))
> # Calculate the compounded wealth from returns
> wealthv <- cbind(wealthv, wealth_trans)
> colnames(wealthv) <- c("Kelly", "Including bid-ask")
> # Plot compounded wealth
> dygraphs::dygraph(wealthv, main="Kelly Strategy With Transaction C
+   dyOptions(colors=c("green", "blue"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Risk Aversion

Risk aversion is the investor preference to avoid losses more than to seek similar percentage gains in wealth.

For example, for a risk averse investor, a 10% loss of wealth is more important than a 10% gain.

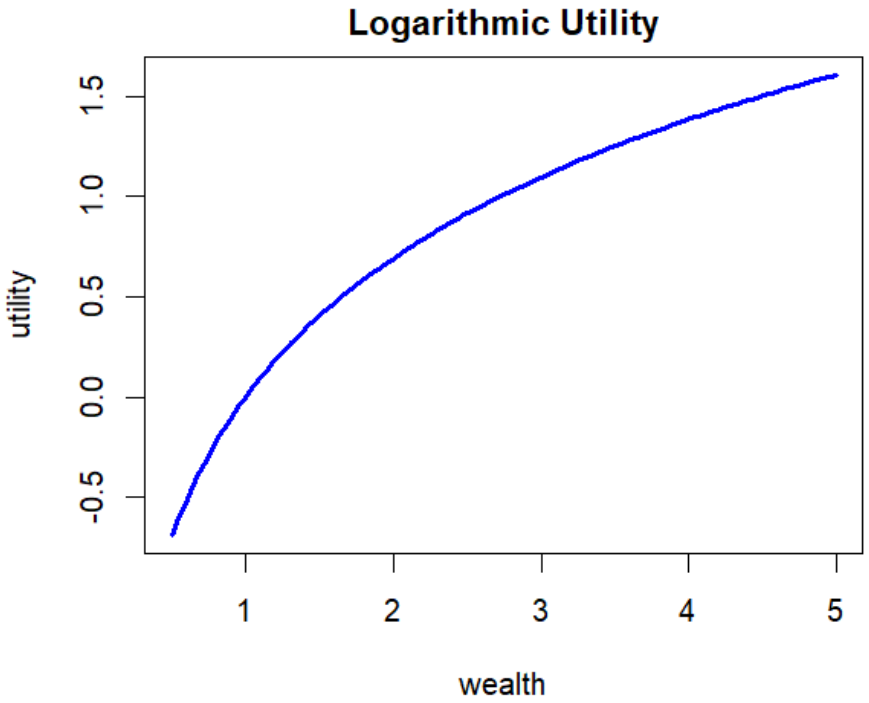
Risk aversion is associated with the *diminishing marginal utility* of the percentage change in wealth Δw.

This manifests itself as a concave utility function, with a negative second derivative $u''(w) < 0$.

For example, the *logarithmic utility* function is concave.

The Arrow-Pratt coefficient of relative risk aversion is proportional to the convexity $u''(w)$ of the utility, and is defined as: $\eta = -\frac{w\, u''(w)}{u'(w)}$.

The relative risk aversion of *logarithmic utility* is equal to one: $\eta = 1$.




```
> # Plot logarithmic utility function
> curve(expr=log, lwd=3, col="blue", xlim=c(0.5, 5),
+ xlab="wealth", ylab="utility",
+ main="Logarithmic Utility")
```

Constant Relative Risk Aversion

It's not a given that all investors have a risk aversion coefficient equal to 1, and other *utility functions* are possible.

The Constant Relative Risk Aversion (*CRRA*) utility function is a generalization of logarithmic utility:

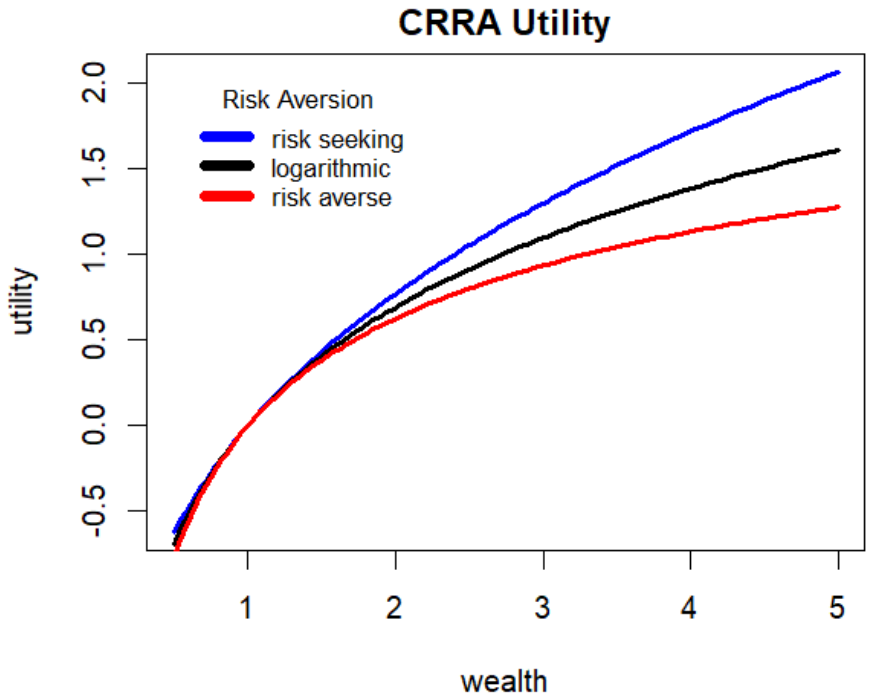
$$u(w) = \frac{w^{1-\eta} - 1}{1 - \eta}$$

Where η is the risk aversion parameter.

The relative risk aversion of the *CRRA* utility function is constant and equal to η.

When the risk aversion parameter is equal to one $\eta = 1$, then the *CRRA* utility function is equal to the logarithmic utility.

In practice, the risk aversion parameter η is not known, and must be estimated through empirical studies.




```
> # Define CRRA utility
> crra_util <- function(w, ra) {
+   (w^(1-ra) - 1)/(1-ra)
+ }  ##  end crra_util
> # Plot utility functions
> curve(expr=crra_util(x, ra=0.7), xlim=c(0.5, 5), lwd=3,
+ xlab="wealth", ylab="utility", main="", col="blue")
> curve(expr=log, add=TRUE, lwd=3)
> curve(expr=crra_util(x, ra=1.3), add=TRUE, lwd=3, col="red")
> # Add title and legend
> title(main="CRRA Utility", line=0.5)
> legend(x="topleft", legend=c("risk seeking", "logarithmic", "risk
+  title="Risk Aversion", inset=0.05, cex=0.8, bg="white", y.intersp
+  lwd=6, lty=1, bty="n", col=c("blue", "black", "red"))
```

The Utility of Lottery Tickets

Lottery tickets are equivalent to binary gambles with a very small probability of winning p, but a very large winning amount a, and a small loss amount b equal to the ticket price.

The expected payout $\mu = p\,a - q\,b$ of most lottery tickets is negative.

So under *logarithmic utility*, the *Kelly ratio* k_r for most lottery tickets is also negative, meaning that investors should not be expected to buy these lottery tickets.

But in reality many people do buy lottery tickets with negative expected payouts, which means that their utility functions are not logarithmic.

The demand for lottery tickets can be explained by assuming a strong demand for positive *skewness*, which is stronger than the demand for a positive payout.

People buy lottery tickets because they want a small chance of a very large payout, even if the average payout is negative.

Without loss of generality we can assume that the lottery ticket price is one dollar $b = 1$, that it pays out a dollars, and that the expected payout is equal to zero: $\mu = p\,a - q\,b = 0$.

Then the probabilities of winning and losing are equal to: $p = \frac{1}{a+1}$ and $q = \frac{a}{a+1}$.

The variance is equal to: $\sigma^2 = p\,q\,(a+1)^2 = a$.

And the *skewness* is equal to:
$\varsigma = \frac{1}{\sigma^3}\left(\frac{a^3}{a+1} - \frac{a}{a+1}\right) = \frac{a-1}{\sqrt{a}}$.

So the positive *skewness* of a lottery ticket increases as the square root of the *betting odds* a, and it can become very large for large *betting odds*.

Investor *Risk Aversion*, *Prudence* and *Temperance*

Investor risk and return preferences depend on the signs of the derivatives of their *utility* function.

Investors with *logarithmic utility* have positive *odd* derivatives ($u'(w) > 0$ and $u'''(w) > 0$) and negative *even* derivatives ($u''(w) < 0$ and $u''''(w) < 0$), which is typical for most other investors as well.

Risk averse investors have a negative second derivative of utility $u''(w) < 0$.

The demand for lottery tickets shows that investors' utility typically has a positive third derivative $u'''(w) > 0$.

Positive *odd* derivatives imply a preference for larger *odd moments* of the change in the wealth distribution (mean, skewness).

Negative *even* derivatives imply a preference for smaller *even moments* (variance, kurtosis).

The preference for smaller *variance* is called *risk aversion*, for larger *skewness* is called *prudence*, and for smaller *kurtosis* is called *temperance*.

The expected change of the *utility* of wealth $\mathbb{E}[\Delta u(w)]$ can be expanded in the moments of the wealth distribution Δw:

$$\mathbb{E}[\Delta u(w)] = u'(w)\mathbb{E}[\Delta w] + \frac{u''(w)}{2}\sigma^2 + \frac{u'''(w)}{3!}\mu 3 + \frac{u''''(w)}{4!}\mu 3$$

Where $\mathbb{E}[\Delta w]$ is the expected change of wealth, $\sigma^2 = \int \Delta w^2\, p(w)\, \mathrm{d}w$ is the *variance* of The change in wealth, and $\mu 3 = \int \Delta w^3\, p(w)\, \mathrm{d}w = \sigma^3 \varsigma$ and $\mu 4 = \int \Delta w^4\, p(w)\, \mathrm{d}w = \sigma^4 \kappa$ are the third and fourth moments, proportional to the *skewness* ς and the *kurtosis* κ.

Investor Preferences and Empirical Return Distributions

The investor preference for higher *returns* and for lower *volatility* is expressed by maximizing the *Sharpe ratio*.

The third and fourth moments of asset returns are usually much smaller than the *variance*, so they typically have a smaller effect on the investor risk and return preferences.

Nevertheless, there is evidence that investors also have significant preferences for positive *skewness* and lower *kurtosis*.

But stock returns typically have negative *skewness* and excess *kurtosis*, the opposite of what investors prefer.

Many investors may prefer positive *skewness*, even at the expense of lower *returns*, similar to the buyers of lottery tickets.

A paper by Amaya asks if the *Realized Skewness Predicts the Cross-Section of Equity Returns?*

But higher moments are hard to estimate accurately from low frequency (daily) returns, which makes empirical investigations more difficult.

```
> # Calculate the VTI returns
> retp <- rutils::etfenv$returns$VTI
> retp <- na.omit(retp)
> # Calculate the higher moments of VTI returns
> c(mean=sum(retp),
+   variance=sum(retp^2),
+   mom3=sum(retp^3),
+   mom4=sum(retp^4))/NROW(retp)
> # Calculate the higher moments of minutely SPY returns
> spy <- HighFreq::SPY[, 4]
> spy <- na.omit(spy)
> spy <- rutils::diffit(log(spy))
> c(mean=sum(spy),
+   variance=sum(spy^2),
+   mom3=sum(spy^3),
+   mom4=sum(spy^4))/NROW(spy)
```

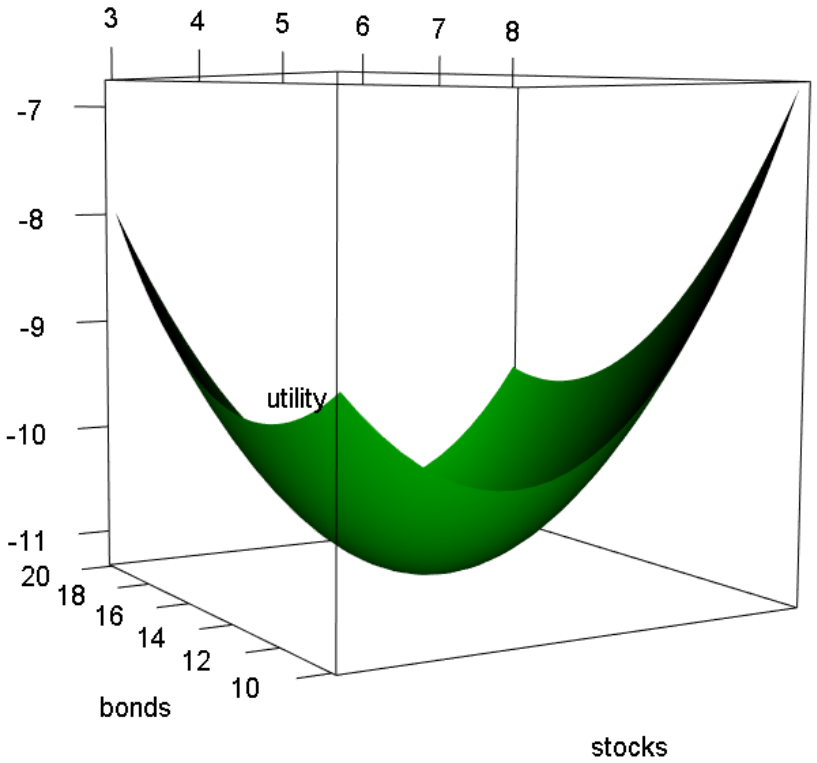
Utility of Stock and Bond Portfolio

The utility u of the stock and bond portfolio with weights *stocku*, *bondu* is equal to:

$$u = \sum_{i=1}^{n} \log(1 + stocku\, r_i^s + bondu\, r_i^b)$$

Where r_i^s, r_i^b are the stock and bond returns.

```
> retp <- na.omit(rutils::etfenv$returns[, c("VTI", "IEF")])
> # Logarithmic utility of stock and bond portfolio
> utilfun <- function(stocku, bondu) {
+   -sum(log(1 + stocku*retp$VTI + bondu*retp$IEF))
+ }  ##  end utilfun
> # Create matrix of utility values
> stocku <- seq(from=3, to=7, by=0.2)
> bondu <- seq(from=12, to=20, by=0.2)
> utilm <- sapply(bondu, function(y) sapply(stocku,
+   function(x) utilfun(x, y)))
> # Set rgl options and load package rgl
> options(rgl.useNULL=TRUE)
> library(rgl)
> # Draw 3d surface plot of utility
> rgl::persp3d(stocku, bondu, utilm, col="green",
+   xlab="stocks", ylab="bonds", zlab="utility")
> # Render the surface plot
> rgl::rglwidget(elementId="plot3drgl")
> # Save the surface plot to png file
> rgl::rgl.snapshot("utility_surface.png")
```



Kelly Optimal Weights

The Kelly optimal stock and bond portfolio weights *stocku*, *bondu* can be calculated by maximizing the utility *u*.

```
> # Approximate Kelly weights
> weightv <- sapply(retp, function(x) mean(x)/var(x))
> # Kelly weight for stocks
> unlist(optimize(f=function(x) utilfun(x, bondu=0), interval=c(1,
> # Kelly weight for bonds
> unlist(optimize(f=function(x) utilfun(x, stocku=0), interval=c(1
> # Vectorized utility of stock and bond portfolio
> utility_vec <- function(weightv) {
+   utilfun(weightv[1], weightv[2])
+ }  ##  end utility_vec
> # Optimize with respect to vector argument
> optiml <- optim(fn=utility_vec, par=c(3, 10),
+           method="L-BFGS-B",
+           upper=c(8, 20), lower=c(2, 5))
> # Exact Kelly weights
> optiml$par
```

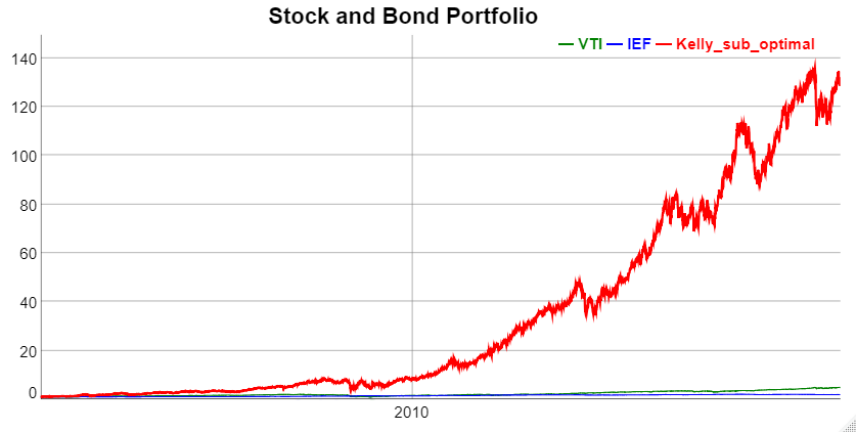
The Kelly optimal weights can be calculated approximately by first calculating the individual stock and bond weights, and then multiplying them by the Kelly weight of the combined portfolio.

```
> # Approximate Kelly weights
> retsport <- (retp %*% weightv)
> drop(mean(retsport)/var(retsport))*weightv
> # Exact Kelly weights
> optiml$par
```

Kelly Optimal Stock and Bond Portfolio

In practice, the Kelly optimal weights under logarithmic utility are too aggressive and they require very active trading, so half-Kelly or even quarter-Kelly weights are used instead.

```
> # Quarter-Kelly sub-optimal weights
> weightv <- optiml$par/4
> # Plot Kelly optimal portfolio
> retp <- cbind(retp, weightv[1]*retp$VTI + weightv[2]*retp$IEF)
> colnames(retp)[3] <- "Kelly_sub_optimal"
> # Calculate the compounded wealth from returns
> wealthv <- cumprod(1 + retp)
> # Plot compounded wealth
> dygraphs::dygraph(wealthv, main="Stock and Bond Portfolio") %>%
+   dyOptions(colors=c("green", "blue", "green")) %>%
+   dySeries("Kelly_sub_optimal", color="red", strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```



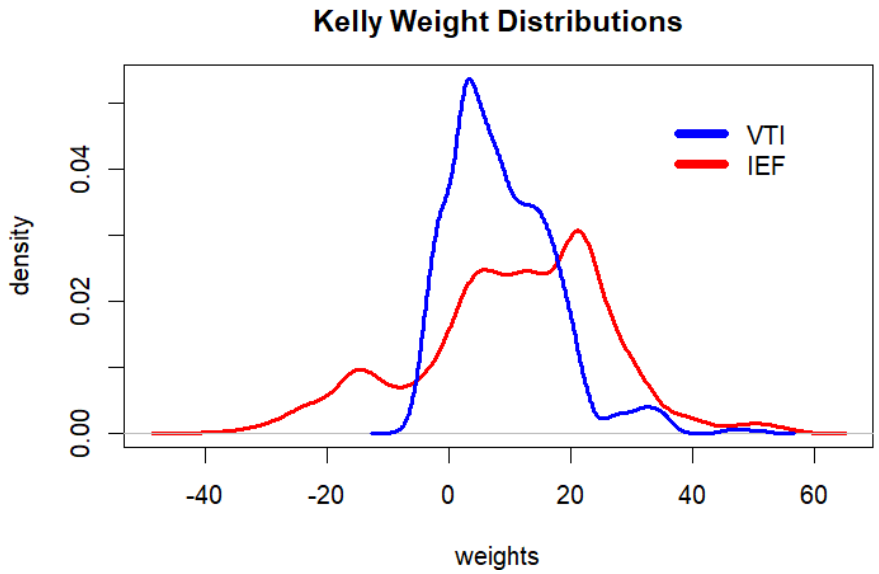
Rolling Kelly Weights

The Kelly weights k_r are calculated daily over a rolling look-back interval:

$$k_r = \frac{\bar{r}_t}{\sigma_t^2}$$

The distribution of the Kelly weights depends on the rolling returns $\bar{r}_t$ and variance σ_t^2.

```
> retp <- na.omit(rutils::etfenv$returns[, c("VTI", "IEF")])
> # Calculate the rolling returns and variance
> lookb <- 200
> var_rolling <- HighFreq::roll_var(retp, lookb)
> weightv <- HighFreq::roll_sum(retp, lookb)/lookb
> weightv <- weightv/var_rolling
> weightv[1, ] <- 1/NCOL(weightv)
> weightv <- zoo::na.locf(weightv)
> sum(is.na(weightv))
> range(weightv)
```



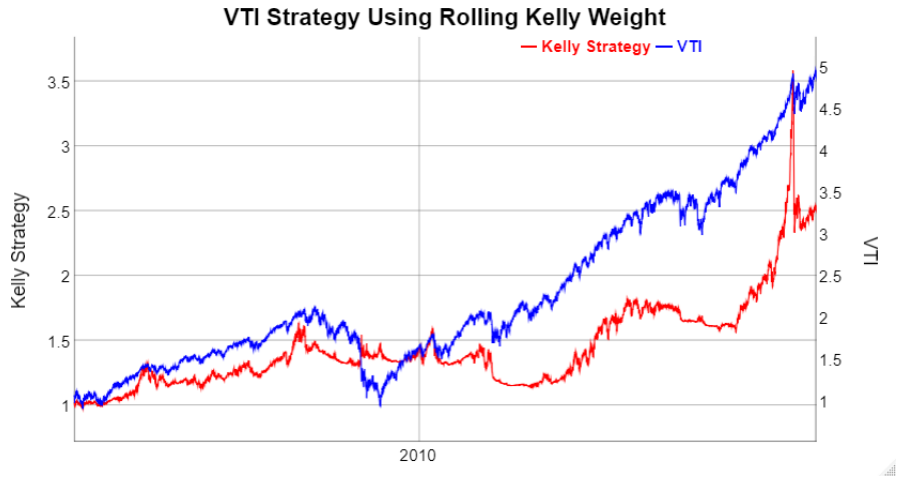

```
> # Plot the weights
> plot(density(retp$IEF), t="l", lwd=3, col="red",
+      xlab="weights", ylab="density",
+      ylim=c(0, max(density(retp$VTI)$y)),
+      main="Kelly Weight Distributions")
> lines(density(retp$VTI), t="l", col="blue", lwd=3)
> legend("topright", legend=c("VTI", "IEF"),
+  inset=0.1, bg="white", lty=1, lwd=6, y.intersp=0.5,
+  col=c("blue", "red"), bty="n")
```

Rolling Kelly Strategy For Stocks

In the rolling Kelly strategy, the leverage of the risky asset k_r changes over time.

The leverage is equal to the updated weight from the previous period.

```
> # Scale and lag the Kelly weights
> weightv <- lapply(weightv, function(x) 10*x/sum(abs(range(x))))
> weightv <- do.call(cbind, weightv)
> weightv <- rutils::lagit(weightv)
> # Calculate the compounded Kelly wealth and VTI
> wealthv <- cbind(cumprod(1 + weightv$VTI*retp$VTI), cumprod(1 + re
> colnames(wealthv) <- c("Kelly Strategy", "VTI")
> dygraphs::dygraph(wealthv, main="VTI Strategy Using Rolling Kelly Weight") %>%
+   dyAxis("y", label="Kelly Strategy", independentTicks=TRUE) %>%
+   dyAxis("y2", label="VTI", independentTicks=TRUE) %>%
+   dySeries(name="Kelly Strategy", axis="y", strokeWidth=1, col="red") %>%
+   dySeries(name="VTI", axis="y2", strokeWidth=1, col="blue")
```



Rolling Kelly Strategy With Transaction Costs

The *margin debt* m_t is proportional to the wealth w_t: $m_t = (k_r - 1)w_t + 1$.

The dollar amount of the *risky asset* traded is equal to the change in *margin*, equal to: $\Delta m_t = \Delta[(k_r - 1)w_t]$.

If the transaction costs are large, then they will reduce the wealth and reduce the dollar amount of the *risky asset* held by the investor.

The transaction costs depend on the change in wealth, and the wealth is decreased by the transaction costs.

So the transaction costs in each time period must be calculated recursively in a loop from the wealth in the past period.

If the transaction costs are much less than the change in wealth $c^r \ll |\Delta w_t|$, then they can be calculated approximately as the absolute value of the change in *margin* m_t^{nc} for a wealth path with no transaction costs:

$$c^r = \frac{\delta}{2} \left| \Delta m_t^{nc} \right|$$

The transaction costs as a percentage of wealth are equal to: c_t / w_t^{nc}, where w_t^{nc} is the wealth assuming no transaction costs.

The wealth of the Kelly Strategy after accounting for the *bid-ask spread* is then equal to:

$$w_t = \prod_{i=1}^{t} (1 + k_r \, r_t - \frac{\delta}{2} \frac{|\Delta m_i^{nc}|}{w_i^{nc}})$$

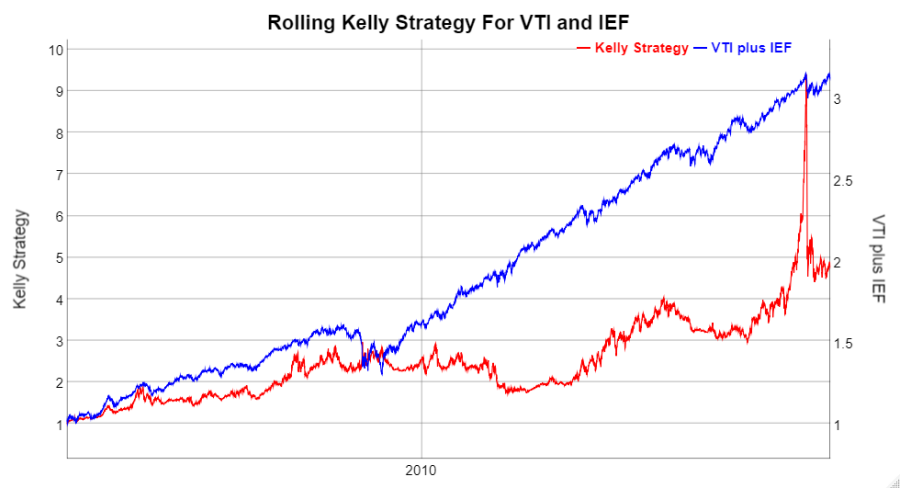
The effect of the *bid-ask spread* is to reduce the effective asset returns by an amount proportional to the *bid-ask spread*.

```
> # bidask equal to 1 bp for liquid ETFs
> bidask <- 0.001
> # Calculate the compounded Kelly wealth and margin
> wealthv <- cumprod(1 + weightv$VTI*retp$VTI)
> marginv <- (retp$VTI - 1)*wealthv + 1
> # Calculate the transaction costs
> costs <- bidask*drop(rutils::diffit(marginv))/2
> wealth_diff <- drop(rutils::diffit(wealthv))
> costs_rel <- ifelse(wealth_diff>0, costs/wealth_diff, 0)
> range(costs_rel)
> hist(costs_rel, breaks=10000, xlim=c(-0.02, 0.02))
> # Scale and lag the transaction costs
> costs <- rutils::lagit(abs(costs)/wealthv)
> # ReCalculate the compounded Kelly wealth
> wealth_trans <- cumprod(1 + retp$VTI*retp$VTI - costs)
> # Plot compounded wealth
> wealthv <- cbind(wealthv, wealth_trans)
> colnames(wealthv) <- c("Kelly", "Including bid-ask")
> dygraphs::dygraph(wealthv, main="Kelly Strategy With Transaction C
+   dyOptions(colors=c("green", "blue"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Rolling Kelly Strategy For Stocks and Bonds

In the rolling Kelly strategy, the leverage of the risky asset k_r changes over time.

The leverage is equal to the updated weight from the previous period.




```
> # Calculate the compounded wealth from returns
> wealthv <- cumprod(1 + rowSums(weightv*retp))
> wealthv <- xts::xts(wealthv, zoo::index(retp))
> quantmod::chart_Series(wealthv, name="Rolling Kelly Strategy For V
> # Calculate the compounded Kelly wealth and VTI
> wealthv <- cbind(wealthv, cumprod(1 + 0.6*retp$IEF + 0.4*retp$VTI)
> colnames(wealthv) <- c("Kelly Strategy", "VTI plus IEF")
> dygraphs::dygraph(wealthv, main="Rolling Kelly Strategy For VTI and IEF") %>%
+   dyAxis("y", label="Kelly Strategy", independentTicks=TRUE) %>%
+   dyAxis("y2", label="VTI plus IEF", independentTicks=TRUE) %>%
+   dySeries(name="Kelly Strategy", axis="y", strokeWidth=1, col="red") %>%
+   dySeries(name="VTI plus IEF", axis="y2", strokeWidth=1, col="blue")
```

The *Alpha* and *Beta* of Stock Returns

The time series of stock returns $r_i - r_f$ in excess of the risk-free rate r_f, can be decomposed into *systematic* returns $\beta(r_m - r_f)$ (where $r_m - r_f$ is the time series of the excess market returns) plus *idiosyncratic* returns $\alpha + \varepsilon_i$ (which are uncorrelated to the market returns):

$$r_i - r_f = \alpha + \beta(r_m - r_f) + \varepsilon_i$$

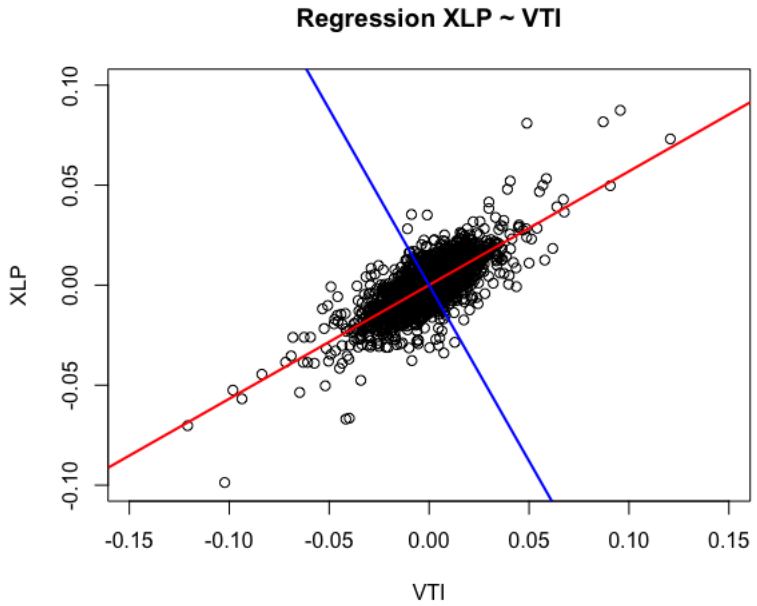
The *alpha* α are the abnormal returns in excess of the risk premium, and ε_i are the regression residuals with zero mean.

The regression residuals ε_i are uncorrelated to the market returns, and can be reduced through portfolio diversification.

The stock β is equal to the ratio of the covariance of the stock returns with the market returns $\sigma_{i,m}$ divided by the variance of the market returns σ_m^2:

$$\beta = \frac{\sigma_{i,m}}{\sigma_m^2} = \rho_{i,m}\frac{\sigma_i}{\sigma_m}$$

Where $\rho_{i,m}$ is the correlation of the stock returns with the market returns, and σ_i and σ_m are the standard deviations of the stock and market returns, respectively.




```
> # Perform regression using formula
> retp <- na.omit(rutils::etfenv$returns[, c("XLP", "VTI")])
> raterf <- 0.03/252
> retp <- (retp - raterf)
> regmod <- lm(XLP ~ VTI, data=retp)
> regsum <- summary(regmod)
> # Get regression coefficients
> coef(regsum)
> # Get alpha and beta
> coef(regsum)[, 1]
> # Plot scatterplot of returns with aspect ratio 1
> plot(XLP ~ VTI, data=rutils::etfenv$returns, main="Regression XLP
+      xlim=c(-0.1, 0.1), ylim=c(-0.1, 0.1), pch=1, col="blue", asp=
> # Add regression line and perpendicular line
> abline(regmod, lwd=2, col="red")
> abline(a=0, b=-1/coef(regsum)[2, 1], lwd=2, col="blue")
```

Capital Asset Pricing Model (*CAPM*)

The *CAPM* model states that the expected return of the stock n: $\mathbb{E}[R_n]$ should be proportional to its beta β_n times the expected excess return of the market $\mathbb{E}[R_m] - r_f$:

$$\mathbb{E}[R_n] = r_f + \beta_n(\mathbb{E}[R_m] - r_f)$$

The *CAPM* model states that stocks should only produce a *systematic* return proportional to their *beta* β values. If a stock has a higher beta then it should also produce higher returns.

The *CAPM* model is not the same as the regression model used to determine the *alpha* α and *beta* β values of stock returns.

In reality, stocks also have a non-zero mean idiosyncratic return *alpha* α_n, in addition to the *systematic* return proportional to their β_n:

$$\mathbb{E}[R_n] = r_f + \alpha_n + \beta_n(\mathbb{E}[R_m] - r_f)$$

```
> library(PerformanceAnalytics)
> # Calculate XLP beta
> PerformanceAnalytics::CAPM.beta(Ra=retp$XLP, Rb=retp$VTI)
> # Or
> betac <- drop(cov(retp$XLP, retp$VTI)/var(retp$VTI))
> betac
> # Calculate XLP alpha
> PerformanceAnalytics::CAPM.alpha(Ra=retp$XLP, Rb=retp$VTI)
> # Or
> alphac <- mean(retp$XLP - betac*retp$VTI)
> # Calculate XLP bull beta
> PerformanceAnalytics::CAPM.beta.bull(Ra=retp$XLP, Rb=retp$VTI)
> # Calculate XLP bear beta
> PerformanceAnalytics::CAPM.beta.bear(Ra=retp$XLP, Rb=retp$VTI)
```

The *Capital Market Line* For a Single Stock

The *CAPM* equation can be expressed in terms of the standard deviation σ_n of the stock returns:

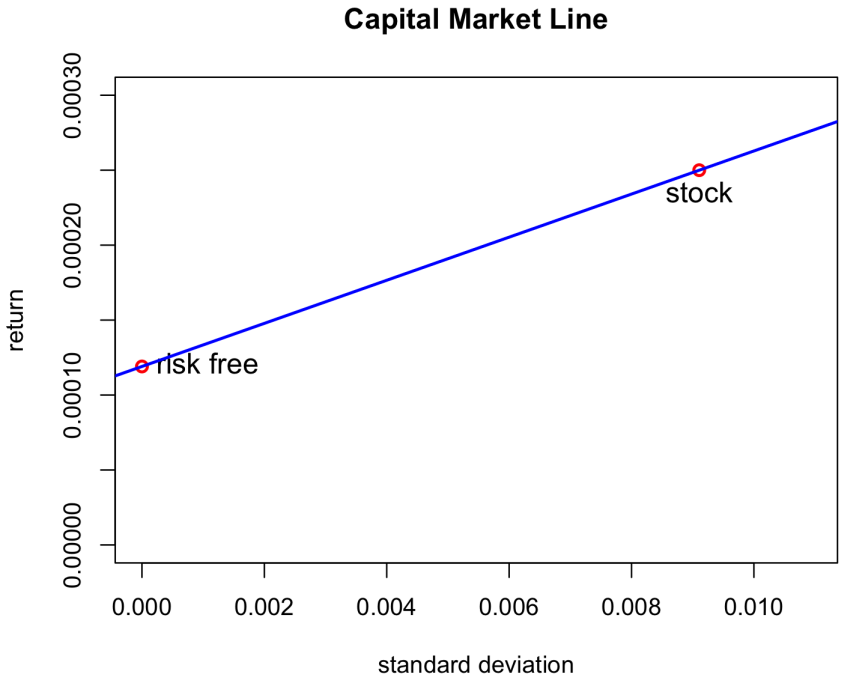
$$\mathbb{E}[R_n] = r_f + \alpha_n + \rho_{n,m}\frac{\sigma_n}{\sigma_m}(\mathbb{E}[R_m] - r_f)$$

This is the *Capital Market Line* (CML), which states that the expected return of the portfolio of the stock and the risk-free asset is proportional to its standard deviation σ_n.

The risk-free asset has zero standard deviation and an expected return equal to the risk-free rate r_f. So when it's added to the stock, the portfolio's return and standard deviation both decrease.

The sum of the portfolio weights is equal to *one*:

$$w_s + w_{rf} = 1$$




```
> # Plot the CML
> rets <- raterf + betac*mean(retp$VTI)
> stdev <- sd(retp$XLP)
> plot(x=c(0, stdev), y=c(raterf, rets), col="red", lwd=2,
+      xlim=c(0, 1.2*stdev), ylim=c(0, 1.2*rets),
+      main="Capital Market Line",
+      xlab="standard deviation", ylab="return")
> text(x=0.0, y=raterf, labels="risk free", pos=4, cex=1.2)
> text(x=stdev, y=rets, labels="stock", pos=1, cex=1.2)
> abline(a=raterf, b=betac*mean(retp$VTI)/stdev, lwd=2, col="blue")
```

The Statistical Significance of *Alpha* and *Beta*

The *t*-statistic (*t*-value) is the ratio of the estimated value divided by its standard error.

The *p*-value is the probability of obtaining values exceeding the *t*-statistic, assuming the *null hypothesis* is true.

A small *p*-value means that the regression coefficients are very unlikely to be zero (given the data).

The *beta* β values of stock returns are very statistically significant, but the *alpha* α values are mostly not significant.

The *p*-value of the *Durbin-Watson* test is large, which indicates that the regression residuals are not autocorrelated.

In practice, the α, β, and the risk-free rate r_f, depend on the time interval of the data, so they're time dependent.

```
> # Get regression coefficients
> coef(regsum)
> # Calculate regression coefficients from scratch
> betac <- drop(cov(retp$XLP, retp$VTI)/var(retp$VTI))
> alphac <- drop(mean(retp$XLP) - betac*mean(retp$VTI))
> c(alphac, betac)
> # Calculate the residuals
> residuals <- (retp$XLP - (alphac + betac*retp$VTI))
> # Calculate the standard deviation of residuals
> nrows <- NROW(residuals)
> residsd <- sqrt(sum(residuals^2)/(nrows - 2))
> # Calculate the standard errors of beta and alpha
> sum2 <- sum((retp$VTI - mean(retp$VTI))^2)
> betasd <- residsd/sqrt(sum2)
> alphasd <- residsd*sqrt(1/nrows + mean(retp$VTI)^2/sum2)
> c(alphasd, betasd)
> # Perform the Durbin-Watson test of autocorrelation of residuals
> lmtest::dwtest(regmod)
```

The *Alpha* and *Beta* of ETF Returns

The *beta* β values of ETF returns are very statistically significant, but the *alpha* α values are mostly not significant.

Some of the ETFs with significant *alpha* α values are the bond ETFs *IEF* and *TLT* (which have performed very well), and the natural resource ETFs *USO* and *DBC* (which have performed very poorly).

```
> retm <- rutils::etfenv$returns
> symbolv <- colnames(retm)
> symbolv <- symbolv[symbolv != "VTI"]
> # Perform regressions and collect statistics
> betam <- sapply(symbolv, function(symbol) {
+ # Specify regression formula
+   formulav <- as.formula(paste(symbol, "~ VTI"))
+ # Perform regression
+   regmod <- lm(formulav, data=retm)
+ # Get regression summary
+   regsum <- summary(regmod)
+ # Collect regression statistics
+   with(regsum,
+     c(beta=coefficients[2, 1],
+ betap=coefficients[2, 4],
+ alpha=coefficients[1, 1],
+ alphap=coefficients[1, 4],
+ dwp=lmtest::dwtest(regmod)$p.value))
+ })  ##  end sapply
> betam <- t(betam)
> # Sort by alphap
> betam <- betam[order(betam[, "alphap"]), ]
```

```
> betam
        beta     betap     alpha   alphap      dwp
IEF  -0.1085 1.41e-125  1.83e-04 0.000621 5.49e-01
VXX  -2.8094  0.00e+00 -1.33e-03 0.000959 5.24e-01
GLD   0.0581  4.09e-06  3.99e-04 0.008808 8.71e-01
USO   0.6830 3.13e-157 -6.85e-04 0.025061 9.34e-02
TLT  -0.2317 1.02e-129  2.47e-04 0.027656 7.24e-01
VEU   0.9747  0.00e+00 -1.98e-04 0.031040 1.00e+00
XLF   1.2479  0.00e+00 -2.30e-04 0.051930 1.00e+00
AIEQ  1.1541 5.26e-230 -2.85e-04 0.150440 9.04e-01
IVE   0.9679  0.00e+00 -6.04e-05 0.204552 1.00e+00
EEM   1.1718  0.00e+00 -1.52e-04 0.242707 1.00e+00
VNQ   1.1367  0.00e+00 -1.87e-04 0.243060 1.00e+00
SVXY  2.1154 1.79e-244 -7.36e-04 0.246519 1.58e-07
XLP   0.5473  0.00e+00  8.98e-05 0.259713 1.00e+00
IWD   0.9652  0.00e+00 -4.78e-05 0.303134 1.00e+00
USMV  0.7043  0.00e+00  5.79e-05 0.371531 9.16e-01
VLUE  0.9686  0.00e+00 -7.33e-05 0.396673 9.86e-01
XLV   0.7276  0.00e+00  6.85e-05 0.414988 7.87e-01
DBC   0.4000 1.01e-199 -1.26e-04 0.421492 9.40e-01
IVW   0.9833  0.00e+00  3.35e-05 0.426340 1.00e+00
QQQ   1.0973  0.00e+00  5.86e-05 0.484433 9.98e-01
XLE   1.0782  0.00e+00 -1.10e-04 0.502282 4.12e-01
DYNF  0.9705  0.00e+00  3.97e-05 0.510397 9.70e-01
XLU   0.6348  0.00e+00  6.75e-05 0.569779 1.00e+00
QUAL  0.9672  0.00e+00  2.13e-05 0.591942 9.98e-01
XLB   1.0196  0.00e+00 -4.42e-05 0.653989 1.00e+00
VTV   0.9377  0.00e+00 -2.25e-05 0.680647 1.00e+00
XLK   1.1113  0.00e+00  3.02e-05 0.722612 9.99e-01
IWF   1.0116  0.00e+00  2.20e-05 0.739039 1.00e+00
XLY   1.0432  0.00e+00  2.32e-05 0.763740 1.00e+00
SPY   0.9859  0.00e+00 -5.71e-06 0.787707 1.00e+00
MTUM  1.0134  0.00e+00  2.37e-05 0.801167 3.47e-02
XLI   0.9964  0.00e+00 -1.05e-05 0.880625 1.00e+00
IWB   0.9829  0.00e+00 -2.43e-06 0.898327 1.00e+00
VYM   0.8572  0.00e+00  9.89e-07 0.988337 1.00e+00
```

The Security Market Line for ETFs

The *Security Market Line* (SML) represents the linear relationship between expected stock returns and their *systematic* risk β.

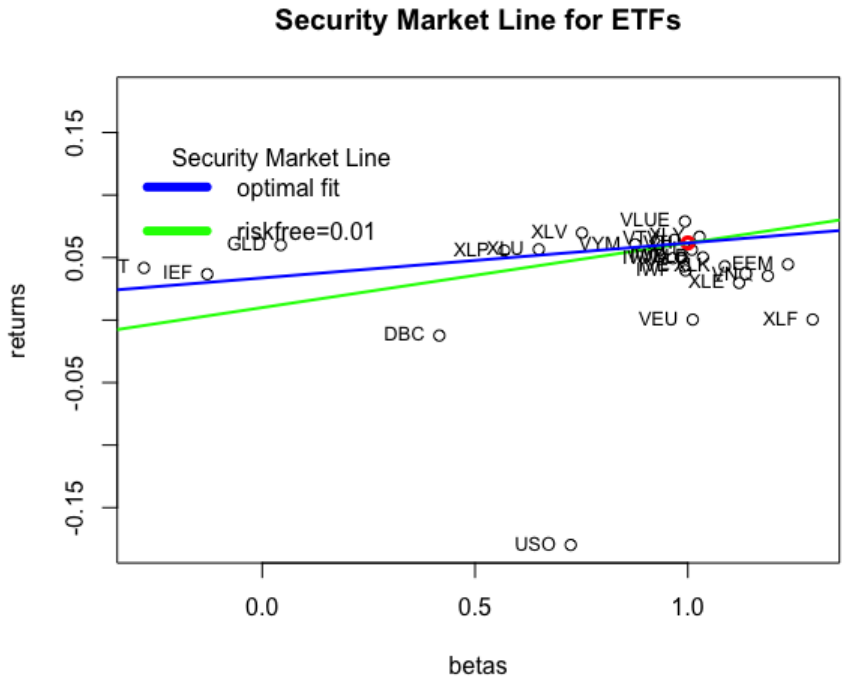
The *SML* depends on the choice of the risk-free rate r_f, with a steeper *SML* line for lower risk-free rates r_f.

All the different *SML* lines pass through the point $(\beta = 1, r = R_m)$ corresponding to the market, and they intersect the y-axis at the risk-free point $(\beta = 0, r = r_f)$.

A scatterplot of asset returns versus their β shows which assets earn a positive α, and which don't.

If an asset lies on the *SML*, then its returns are mostly *systematic*, and its α is equal to zero.

Assets above the *SML* have a positive α, and those below have a negative α.




```
> symbolv <- rownames(betam)
> betac <- betam[-match(c("VXX", "SVXY", "MTUM", "USMV", "QUAL"), s
> betac <- c(1, betac)
> names(betac)[1] <- "VTI"
> retsann <- sapply(retp[, names(betac)], PerformanceAnalytics::Ret
> # Plot scatterplot of returns vs betas
> minrets <- min(retsann)
> plot(retsann ~ betac, xlab="betas", ylab="returns",
+      ylim=c(minrets, -minrets), main="Security Market Line for E
> retvti <- retsann["VTI"]
> points(x=1, y=retvti, col="red", lwd=3, pch=21)
> # Plot Security Market Line
> raterf <- 0.01
```

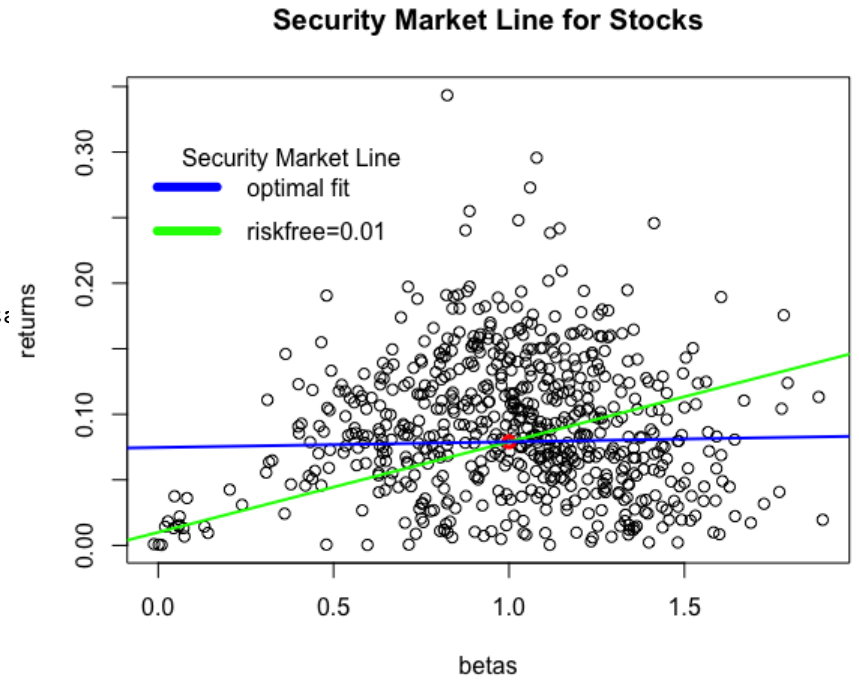
```
> # Add labels
> text(x=betac, y=retsann, labels=names(betac), pos=2, cex=0.8)
> # Find optimal risk-free rate by minimizing residuals
> rss <- function(raterf) {
+   sum((retsann - raterf - betac*(retvti-raterf))^2)
+ }  ##  end rss
> optimrss <- optimize(rss, c(-1, 1))
> raterf <- optimrss$minimum
> # Or simply
> retsadj <- (retsann - retvti*betac)
> betadj <- (1-betac)
> raterf <- sum(retsadj*betadj)/sum(betadj^2)
> abline(a=raterf, b=(retvti-raterf), col="blue", lwd=2)
> legend(x="topleft", bty="n", title="Security Market Line",
+  legend=c("optimal fit", "raterf=0.01"),
+  y.intersp=0.5, cex=1.0, lwd=6, lty=1, col=c("blue", "green"))
```

The Security Market Line for Stocks

The best fitting *Security Market Line* (SML) for stocks is almost flat, which shows that stocks with higher β don't earn higher returns.

This is called the *low beta anomaly*.

```
> # Load S&P500 constituent stock returns
> load("/Users/jerzy/Develop/lecture_slides/data/sp500_returns.RDat
> retvti <- na.omit(rutils::etfenv$returns$VTI)
> retp <- retstock[index(retvti), ]
> nrows <- NROW(retp)
> # Calculate stock betas
> betac <- sapply(retp, function(x) {
+   retp <- na.omit(cbind(x, retvti))
+   drop(cov(retp[, 1], retp[, 2])/var(retp[, 2]))
+ })  ##  end sapply
> mean(betac)
> # Calculate annual stock returns
> retsann <- retp
> retsann[1, ] <- 0
> retsann <- zoo::na.locf(retsann, na.rm=FALSE)
> retsann <- 252*sapply(retsann, sum)/nrows
> # Remove stocks with zero returns
> sum(retsann == 0)
> betac <- betac[retsann > 0]
> retsann <- retsann[retsann > 0]
> retvti <- 252*mean(retvti)
> # Plot scatterplot of returns vs betas
> plot(retsann ~ betac, xlab="betas", ylab="returns",
+      main="Security Market Line for Stocks")
> points(x=1, y=retvti, col="red", lwd=3, pch=21)
> # Plot Security Market Line
> raterf <- 0.01
> abline(a=raterf, b=(retvti-raterf), col="green", lwd=2)
```




```
> # Find optimal risk-free rate by minimizing residuals
> retsadj <- (retsann - retvti*betac)
> betadj <- (1-betac)
> raterf <- sum(retsadj*betadj)/sum(betadj^2)
> abline(a=raterf, b=(retvti-raterf), col="blue", lwd=2)
> legend(x="topleft", bty="n", title="Security Market Line",
+  legend=c("optimal fit", "raterf=0.01"),
+  y.intersp=0.5, cex=1.0, lwd=6, lty=1, col=c("blue", "green"))
```

The Capital Market Line And The Security Market Line

The CAPM formula can be applied in three different contexts.

First, as a regression of the time series of stock returns versus the time series of market returns, to determine the alpha α and beta β coefficients:

$$r_i = r_f + \alpha + \beta(r_m - r_f) + \varepsilon_i$$

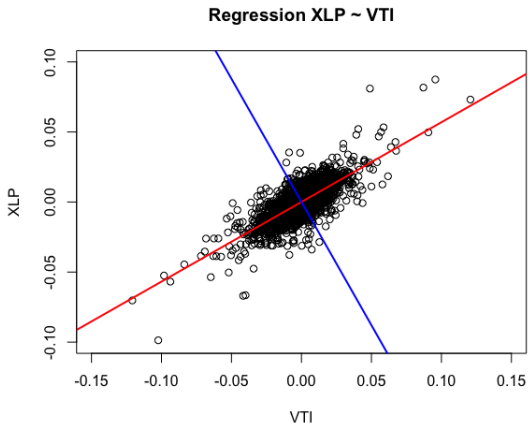
Second, as the Security Market Line (SML), which relates the expected stock returns to their betas β_n:

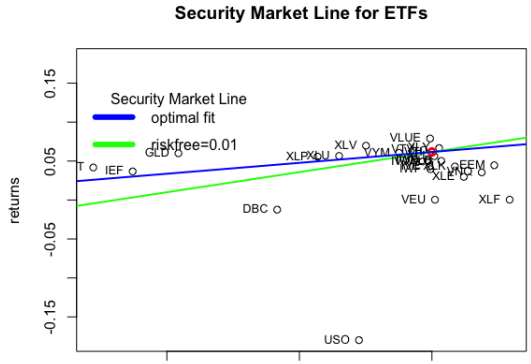
$$\mathbb{E}[R_n] = r_f + \alpha_n + \beta_n(\mathbb{E}[R_m] - r_f)$$

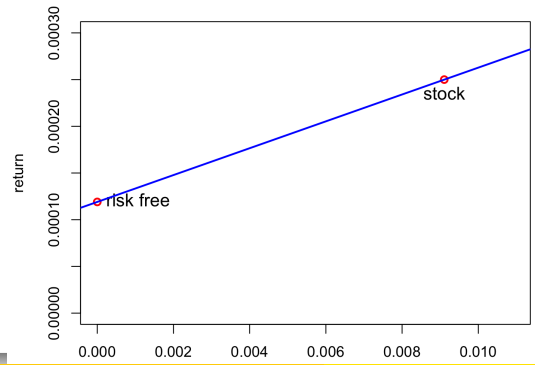
Third, as the Capital Market Line (CML), which relates the return of the portfolio of a stock and the risk-free asset to its standard deviation σ_n.

$$\mathbb{E}[R_n] = r_f + \alpha_n + \rho_{n,m}\frac{\sigma_n}{\sigma_m}(\mathbb{E}[R_m] - r_f)$$

Note that the risk-free asset has zero standard deviation, and an expected return equal to r_f.







The Treynor and Information Ratios

The *Treynor* ratio measures the excess returns per unit of the *systematic* risk *beta* β, and is equal to the excess returns (over a risk-free rate) divided by the β:

$$T_r = \frac{E[R - r_f]}{\beta}$$

The *Treynor* ratio is similar to the *Sharpe* ratio, with the difference that its denominator represents only *systematic* risk, not total risk.

The *Information* ratio is equal to the excess returns (over a benchmark) divided by the *tracking error* (standard deviation of excess returns):

$$I_r = \frac{E[R - R_b]}{\sqrt{\sum_{i=1}^{n}(R_i - R_{i,b})^2}}$$

The *Information* ratio measures the amount of outperformance versus the benchmark, and the consistency of outperformance.

```
> retp <- na.omit(rutils::etfenv$returns[, c("XLP", "VTI")])
> library(PerformanceAnalytics)
> # Calculate XLP Treynor ratio
> TreynorRatio(Ra=retp$XLP, Rb=retp$VTI)
[1] 0.116
> # Calculate XLP Information ratio
> InformationRatio(Ra=retp$XLP, Rb=retp$VTI)
[1] -0.0833
```

CAPM Summary Statistics

`PerformanceAnalytics::table.CAPM()` calculates the *beta* β and *alpha* α values, the *Treynor* ratio, and other performance statistics.

```
> PerformanceAnalytics::table.CAPM(Ra=retm[, c("XLP", "XLF")],
+                           Rb=retm$VTI, scale=252)
                   XLP to VTI XLF to VTI
Alpha                  0.0001    -0.0002
Beta                   0.5473     1.2479
Alpha Robust           0.0001    -0.0003
Beta Robust            0.5450     1.0756
Beta+                  0.5626     1.3164
Beta-                  0.5636     1.3227
Beta+ Robust           0.5444     1.0934
Beta- Robust           0.5516     1.1138
R-squared              0.5263     0.7233
R-squared Robust       0.4249     0.6194
Annualized Alpha       0.0229    -0.0564
Correlation            0.7255     0.8505
Correlation p-value    0.0000     0.0000
Tracking Error         0.1320     0.1553
Active Premium        -0.0110    -0.0595
Information Ratio     -0.0833    -0.3828
Treynor Ratio          0.1001     0.0149
```

```
> capmstats <- table.CAPM(Ra=retm[, symbolv], Rb=retm$VTI, scale=252)
> colv <- strsplit(colnames(capmstats), split=" ")
> colv <- do.call(cbind, colv)[1, ]
> colnames(capmstats) <- colv
> capmstats <- t(capmstats)
> capmstats <- capmstats[, -1]
> colv <- colnames(capmstats)
> whichv <- match(c("Annualized Alpha", "Information Ratio", "Treynor Rat
> colv[whichv] <- c("Alpha", "Information", "Treynor")
> colnames(capmstats) <- colv
> capmstats <- capmstats[order(capmstats[, "Alpha"], decreasing=TRUE), ]
> # Copy capmstats into etfenv and save to .RData file
> etfenv <- rutils::etfenv
> etfenv$capmstats <- capmstats
```

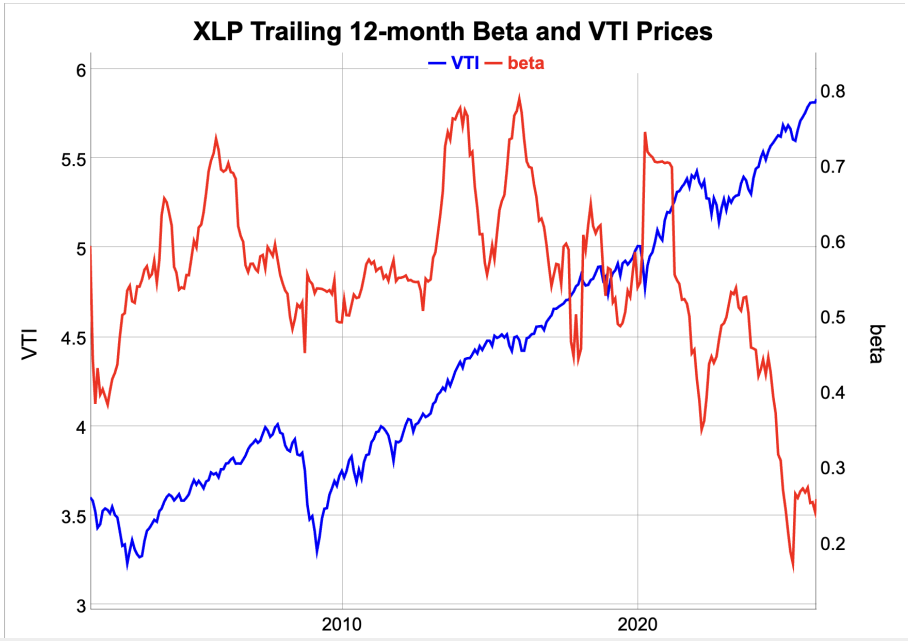
```
> rutils::etfenv$capmstats[, c("Beta", "Alpha", "Information"
        Beta   Alpha Information Treynor
GLD   0.0515  0.1066      0.0301  1.8482
TLT  -0.2350  0.0641     -0.2470 -0.1144
IEF  -0.1101  0.0471     -0.2791 -0.2940
XLU   0.6116  0.0246     -0.0473  0.0938
XLV   0.7332  0.0224      0.0221  0.0930
XLP   0.5433  0.0208     -0.0746  0.1009
XLY   1.0126  0.0120      0.0518  0.0706
XLI   0.9657  0.0116      0.0535  0.0733
USMV  0.7231  0.0102     -0.3505  0.1489
QQQ   1.1895  0.0040      0.0106  0.0546
VTI   0.9948  0.0032      0.1023  0.0749
IVW   0.9955  0.0025      0.0114  0.0654
IWB   0.9786  0.0022      0.0185  0.0655
XLB   0.9443  0.0021     -0.0825  0.0563
DYNF  0.9939  0.0014     -0.0008  0.1343
MTUM  1.0322  0.0000     -0.0030  0.1241
IWD   0.9422 -0.0005     -0.0852  0.0624
VYM   0.8633 -0.0008     -0.1715  0.0839
QUAL  0.9889 -0.0013     -0.1010  0.1206
XLE   0.9806 -0.0024     -0.1313  0.0377
XLK   1.1601 -0.0026     -0.0326  0.0527
IWF   1.0323 -0.0030     -0.0683  0.0565
IVE   0.9551 -0.0047     -0.1449  0.0577
VTV   0.9440 -0.0056     -0.1879  0.0787
VLUE  0.9790 -0.0229     -0.3956  0.0985
DBC   0.4003 -0.0316     -0.4631 -0.0228
EEM   1.1803 -0.0362     -0.2442  0.0479
XLF   1.2158 -0.0405     -0.2882  0.0152
VNQ   1.1326 -0.0449     -0.3073  0.0276
VEU   0.9861 -0.0505     -0.6122  0.0266
AIEQ  1.1493 -0.0712     -0.7815  0.1116
USO   0.6903 -0.1598     -0.7079 -0.2287
SVXY  2.1465 -0.1774         NaN     NaN
VXX  -2.8726 -0.2765     -0.9328  0.2135
```

Trailing Stock Beta Over Time

The trailing beta of *XLP* versus *VTI* changes over time, with lower beta in periods of stock selloffs.

The function `roll_reg()` from package *HighFreq* performs trailing regressions in `C++`, so it's much faster than equivalent `R` code.

```
> # Calculate XLP and VTI returns
> retp <- na.omit(rutils::etfenv$returns[, c("XLP", "VTI")])
> # Calculate monthly end points
> endd <- rutils::calc_endpoints(retp, interval="months")[-1]
> # Calculate start points from look-back interval
> lookb <- 12  ##  Look back 12 months
> startp <- c(rep(1, lookb), endd[1:(NROW(endd)-lookb)])
> head(cbind(endd, startp), lookb+2)
> # Calculate trailing beta regressions every month in R
> formulav <- XLP ~ VTI  ##  Specify regression formula
> betar <- sapply(1:NROW(endd), FUN=function(tday) {
+     datav <- retp[startp[tday]:endd[tday], ]
+     ##  coef(lm(formulav, data=datav))[2]
+     drop(cov(datav$XLP, datav$VTI)/var(datav$VTI))
+ })  ##  end sapply
> # Calculate trailing betas using RcppArmadillo
> controll <- HighFreq::param_reg()
> reg_stats <- HighFreq::roll_reg(respv=retp$XLP, predm=retp$VTI,
+   startp=(startp-1), endd=(endd-1), controll=controll)
> betac <- reg_stats[, 1]
> all.equal(betac, betar)
> # Compare the speed of RcppArmadillo with R code
> library(microbenchmark)
> summary(microbenchmark(
+   Rcpp=HighFreq::roll_reg(respv=retp$XLP, predm=retp$VTI, startp=(startp-1), endd=(endd-1), controll=controll),
+   Rcode=sapply(1:NROW(endd), FUN=function(tday) {
+     datav <- retp[startp[tday]:endd[tday], ]
+     drop(cov(datav$XLP, datav$VTI)/var(datav$VTI))
+   }),
+   times=10))[, c(1, 4, 5)]  ##  end microbenchmark summary
```




```
> # dygraph plot of trailing XLP beta and VTI prices
> datev <- zoo::index(retp[endd, ])
> pricev <- log(rutils::etfenv$prices$VTI[datev])
> datav <- cbind(pricev, betac)
> colnames(datav)[2] <- "beta"
> colv <- colnames(datav)
> dygraphs::dygraph(datav, main="XLP Trailing 12-month Beta and VTI
+   dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+   dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+   dySeries(name=colv[1], axis="y", col="blue", strokeWidth=2) %>%
+   dySeries(name=colv[2], axis="y2", col="red", strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Trailing EMA Stock Beta

The trailing beta β of a stock with returns r_t with respect to a stock index with returns R_t can be updated using these recursive EMA formulas with the decay factor λ:

$$\bar{r}_t = \lambda \bar{r}_{t-1} + (1-\lambda) r_t$$

$$\bar{R}_t = \lambda \bar{R}_{t-1} + (1-\lambda) R_t$$

$$\sigma_t^2 = \lambda \sigma_{t-1}^2 + (1-\lambda)(R_t - \bar{R}_t)^2$$

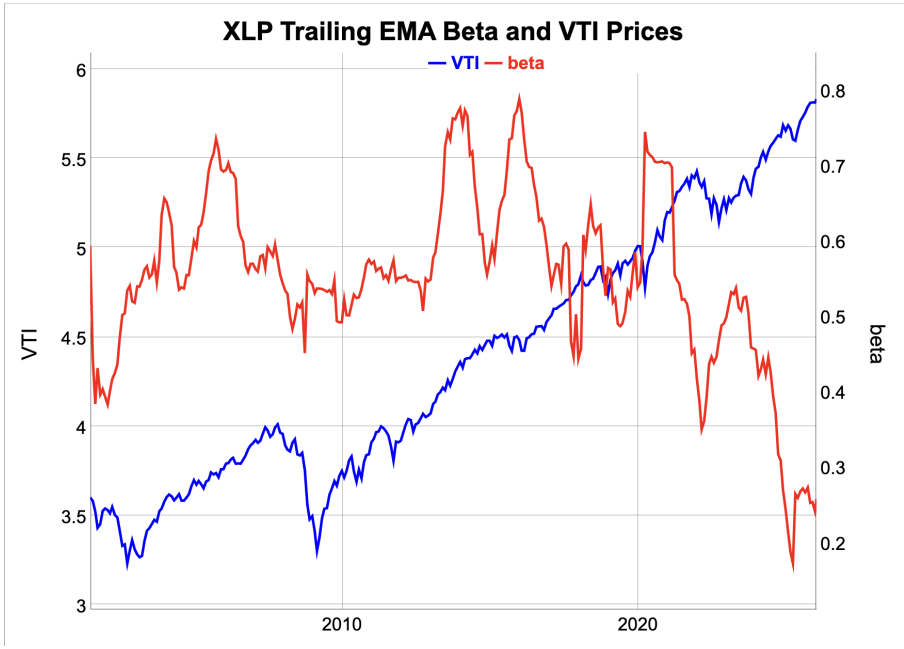
$$\text{cov}_t = \lambda \, \text{cov}_{t-1} + (1-\lambda)(r_t - \bar{r}_t)(R_t - \bar{R}_t)$$

$$\beta_t = \frac{\text{cov}_t}{\sigma_t^2}$$

The parameter λ determines the rate of decay of the weight of past returns. If λ is close to 1 then the decay is weak and past returns have a greater weight, and the trailing mean values have a stronger dependence on past returns. This is equivalent to a long look-back interval. And vice versa if λ is close to 0.

The function `HighFreq::run_covar()` calculates the trailing variances, covariances, and means of two *time series*.

```
> # Calculate the trailing betas
> lambdaf <- 0.99
> covarv <- HighFreq::run_covar(retp, lambdaf)
> betac <- covarv[, 1]/covarv[, 3]
```




```
> # dygraph plot of trailing XLP beta and VTI prices
> datav <- cbind(pricev, betac[endd])[-(1:11)] ##  Remove warmup per
> colnames(datav)[2] <- "beta"
> colv <- colnames(datav)
> dygraphs::dygraph(datav, main="XLP Trailing EMA Beta and VTI Price
+   dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+   dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+   dySeries(name=colv[1], axis="y", col="blue", strokeWidth=2) %>%
+   dySeries(name=colv[2], axis="y2", col="red", strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Package *Rcpp* for Calling C++ Programs from R

The package *Rcpp* allows calling C++ functions from R, by compiling the C++ code and creating R functions.

Rcpp functions are R functions that were compiled from C++ code using package *Rcpp*.

Rcpp functions are much faster than code written in R, so they're suitable for large numerical calculations.

The package *Rcpp* relies on *Rtools* for compiling the C++ code:

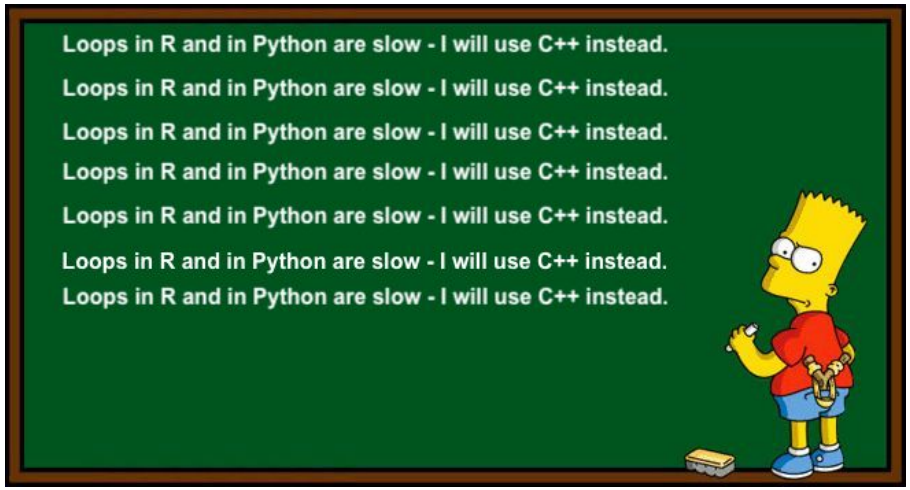
https://cran.r-project.org/bin/windows/Rtools/

You can learn more about the package *Rcpp* here:

http://adv-r.had.co.nz/Rcpp.html

http://www.rcpp.org/

http://gallery.rcpp.org/




```
> # Verify that Rtools or XCode are working properly:
> devtools::find_rtools()  ##  Under Windows
> devtools::has_devel()
> # Install the packages Rcpp and RcppArmadillo
> install.packages(c("Rcpp", "RcppArmadillo"))
> # Load package Rcpp
> library(Rcpp)
> # Get documentation for package Rcpp
> # Get short description
> packageDescription("Rcpp")
> # Load help page
> help(package="Rcpp")
> # List all datasets in "Rcpp"
> data(package="Rcpp")
> # List all objects in "Rcpp"
> ls("package:Rcpp")
> # Remove Rcpp from search path
> detach("package:Rcpp")
```

Function cppFunction() for Compiling C++ code

The function cppFunction() compiles C++ code into an R function.

The function cppFunction() creates an R function only for the current R session, and it must be recompiled for every new R session.

The function sourceCpp() compiles C++ code contained in a file into R functions.

```
> # Define Rcpp function
> Rcpp::cppFunction("
+   int times_two(int x)
+     { return 2 * x;}
+   ")  ##  end cppFunction
> # Run Rcpp function
> times_two(3)
> # Source Rcpp functions from file
> Rcpp::sourceCpp(file="/Users/jerzy/Develop/lecture_slides/scripts/
> # Multiply two numbers
> mult_rcpp(2, 3)
> mult_rcpp(1:3, 6:4)
> # Multiply two vectors
> mult_vec_rcpp(2, 3)
> mult_vec_rcpp(1:3, 6:4)
```

Performing Loops in *Rcpp Sugar*

Loops written in *Rcpp* can be two orders of magnitude faster than loops in R!

Rcpp Sugar allows using R-style vectorized syntax in *Rcpp* code.

```
> # Define Rcpp function with loop
> Rcpp::cppFunction("
+ double inner_mult(NumericVector x, NumericVector y) {
+ int xsize = x.size();
+ int ysize = y.size();
+ if (xsize != ysize) {
+     return 0;
+   } else {
+     double total = 0;
+     for(int i = 0; i < xsize; ++i) {
+ total += x[i] * y[i];
+   }
+   return total;
+   }
+ }")  ##  end cppFunction
> # Run Rcpp function
> inner_mult(1:3, 6:4)
> inner_mult(1:3, 6:3)
> # Define Rcpp Sugar function with loop
> Rcpp::cppFunction("
+ double inner_sugar(NumericVector x, NumericVector y) {
+   return sum(x * y);
+ }")  ##  end cppFunction
> # Run Rcpp Sugar function
> inner_sugar(1:3, 6:4)
> inner_sugar(1:3, 6:3)
```

```
> # Define R function with loop
> inner_multr <- function(x, y) {
+     sumv <- 0
+     for(i in 1:NROW(x)) {
+ sumv <- sumv + x[i] * y[i]
+     }
+     sumv
+ }  ##  end inner_multr
> # Run R function
> inner_multr(1:3, 6:4)
> inner_multr(1:3, 6:3)
> # Compare speed of Rcpp and R
> library(microbenchmark)
> summary(microbenchmark(
+   rcode=inner_multr(1:10000, 1:10000),
+   innerp=1:10000 %*% 1:10000,
+   Rcpp=inner_mult(1:10000, 1:10000),
+   sugar=inner_sugar(1:10000, 1:10000),
+   times=10))[, c(1, 4, 5)]
```

Simulating Ornstein-Uhlenbeck Process Using *Rcpp*

Simulating the Ornstein-Uhlenbeck Process in *Rcpp* is about 30 times faster than in R!

```
> # Define Ornstein-Uhlenbeck function in R
> sim_our <- function(nrows=1000, priceq=5.0,
+               volat=0.01, theta=0.01) {
+   retp <- numeric(nrows)
+   pricev <- numeric(nrows)
+   pricev[1] <- priceq
+   for (i in 2:nrows) {
+     retp[i] <- theta*(priceq - pricev[i-1]) + volat*rnorm(1)
+     pricev[i] <- pricev[i-1] + retp[i]
+   }  ##  end for
+   pricev
+ }  ##  end sim_our
> # Simulate Ornstein-Uhlenbeck process in R
> priceq <- 5.0; sigmav <- 0.01
> thetav <- 0.01; nrows <- 1000
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")  ##
> ousim <- sim_our(nrows, priceq=priceq, volat=sigmav, theta=theta
```

```
> # Define Ornstein-Uhlenbeck function in Rcpp
> Rcpp::cppFunction("
+ NumericVector sim_oucpp(double priceq,
+                   double volat,
+                   double thetav,
+                   NumericVector innov) {
+   int nrows = innov.size();
+   NumericVector pricev(nrows);
+   NumericVector retv(nrows);
+   pricev[0] = priceq;
+   for (int it = 1; it < nrows; it++) {
+     retv[it] = thetav*(priceq - pricev[it-1]) + volat*innov[it-1];
+     pricev[it] = pricev[it-1] + retv[it];
+   }  // end for
+   return pricev;
+ }")  ##  end cppFunction
> # Simulate Ornstein-Uhlenbeck process in Rcpp
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")  ##  R
> oucpp <- sim_oucpp(priceq=priceq,
+   volat=sigmav, theta=thetav, innov=rnorm(nrows))
> all.equal(ousim, oucpp)
> # Compare speed of Rcpp and R
> library(microbenchmark)
> summary(microbenchmark(
+   rcode=sim_our(nrows, priceq=priceq, volat=sigmav, theta=thetav),
+   Rcpp=sim_oucpp(priceq=priceq, volat=sigmav, theta=thetav, innov=
+   times=10))[, c(1, 4, 5)]
```

Rcpp Attributes

Rcpp attributes are instructions for the C++ compiler, embedded in the *Rcpp* code as C++ comments, and preceded by the "//" symbol.

The `Rcpp::depends` attribute specifies additional C++ library dependencies.

The `Rcpp::export` attribute specifies that a function should be exported to R, where it can be called as an R function.

Only functions which are preceded by the `Rcpp::export` attribute are exported to R.

The function `sourceCpp()` compiles C++ code contained in a file into R functions.

```
// Rcpp header with information for C++ compiler
#include <Rcpp.h> // include Rcpp C++ header files
using namespace Rcpp; // use Rcpp C++ namespace

// The function sim_oucpp() simulates an Ornstein-Uhlenb
// export the function roll_maxmin() to R
// [[Rcpp::export]]
NumericVector sim_oucpp(double priceq,
                        double volat,
                        double thetav,
                        NumericVector innov) {
  int(nrows = innov.size();
  NumericVector pricev*nrows);
  NumericVector retp*nrows);
  pricev[0] = priceq;
  for (int it = 1; it < nrows; it++) {
    retp[it] = thetav*(priceq - pricev[it-1]) + volat*in
    pricev[it] = pricev[it-1] + retp[it];
  }  // end for
  return pricev;
}  // end sim_oucpp
```

```
> # Source Rcpp function for Ornstein-Uhlenbeck process from file
> Rcpp::sourceCpp(file="/Users/jerzy/Develop/lecture_slides/scripts/
> # Simulate Ornstein-Uhlenbeck process in Rcpp
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")  ##  Reset random numbers
> oucpp <- sim_oucpp(priceq=priceq,
+   volat=sigmav,
+   theta=thetav,
+   innov=rnorm(nrows))
> all.equal(ousim, oucpp)
> # Compare speed of Rcpp and R
> library(microbenchmark)
> summary(microbenchmark(
+   rcode=sim_our(nrows, priceq=priceq, volat=sigmav, theta=thetav),
+   Rcpp=sim_oucpp(priceq=priceq, volat=sigmav, theta=thetav, innov=rnorm(nrows)),
+   times=10))[, c(1, 4, 5)]
```

Generating Random Numbers Using Logistic Map in *Rcpp*

The *logistic map* in *Rcpp* is about seven times faster than the loop in R, and even slightly faster than the standard `runif()` function in R!

```
> # Calculate uniformly distributed pseudo-random sequence
> unifun <- function(seedv, nrows=10) {
+   datav <- numeric(nrows)
+   datav[1] <- seedv
+   for (i in 2:nrows) {
+     datav[i] <- 4*datav[i-1]*(1-datav[i-1])
+   }  ##  end for
+   acos(1-2*datav)/pi
+ }  ##  end unifun
>
> # Source Rcpp functions from file
> Rcpp::sourceCpp(file="/Users/jerzy/Develop/lecture_slides/scripts,
> # Microbenchmark Rcpp code
> library(microbenchmark)
> summary(microbenchmark(
+   rcode=runif(1e5),
+   rloop=unifun(0.3, 1e5),
+   Rcpp=unifuncpp(0.3, 1e5),
+   times=10))[, c(1, 4, 5)]
```

```
// Rcpp header with information for C++ compiler
#include <Rcpp.h> // include Rcpp C++ header files
using namespace Rcpp; // use Rcpp C++ namespace

// This is a simple example of exporting a C++ function
// You can source this function into an R session using
// function Rcpp::sourceCpp()
// (or via the Source button on the editor toolbar).
// Learn more about Rcpp at:
//
//   http://www.rcpp.org/
//   http://adv-r.had.co.nz/Rcpp.html
//   http://gallery.rcpp.org/

// function unifun() produces a vector of
// uniformly distributed pseudo-random numbers
// [[Rcpp::export]]
NumericVector unifuncpp(double seedv, int(nrows) {
// define pi
static const double pi = 3.14159265;
// allocate output vector
  NumericVector datav(nrows);
// initialize output vector
  datav[0] = seedv;
// perform loop
  for (int i=1; i < nrows; ++i) {
    datav[i] = 4*datav[i-1]*(1-datav[i-1]);
  }  // end for
// rescale output vector and return it
  return acos(1-2*datav)/pi;
}
```

Package *RcppArmadillo* for Fast Linear Algebra

The package *RcppArmadillo* allows calling from R the high-level *Armadillo* C++ linear algebra library.

Armadillo provides ease of use and speed, with syntax similar to *Matlab*.

RcppArmadillo functions are often faster than even compiled R functions, because they use better optimized C++ code:
http://arma.sourceforge.net/speed.html

You can learn more about *RcppArmadillo*:
http://arma.sourceforge.net/
http://dirk.eddelbuettel.com/code/rcpp.armadillo.html
https://cran.r-project.org/web/packages/\emph{RcppArmadillo}/index.html
https://github.com/RcppCore/\emph{RcppArmadillo}

```
> library(RcppArmadillo)
> # Source Rcpp functions from file
> Rcpp::sourceCpp(file="/Users/jerzy/Develop/lecture_slides/scripts
> vec1 <- runif(1e5)
> vec2 <- runif(1e5)
> inner_vec(vec1, vec2)
> vec1 %*% vec2
```

```
// Rcpp header with information for C++ compiler
#include <RcppArmadillo.h>
using namespace Rcpp;
using namespace arma;
// [[Rcpp::depends(RcppArmadillo)]]

// The function inner_vec() calculates the inner (dot) p
// It uses \emph{RcppArmadillo}.
//' @export
// [[Rcpp::export]]
double inner_vec(arma::vec vec1, arma::vec vec2) {
  return arma::dot(vec1, vec2);
}  // end inner_vec

// The function inner_mat() calculates the inner (dot) p
// with two vectors.
// It accepts pointers to the matrix and vectors, and re
// It uses \emph{RcppArmadillo}.
//' @export
// [[Rcpp::export]]
double inner_mat(const arma::vec& vecv2, const arma::mat
  return arma::as_scalar(trans(vecv2) * (matv * vecv1));
}  // end inner_mat
```

```
> # Microbenchmark \emph{RcppArmadillo} code
> summary(microbenchmark(
+   rcpp = inner_vec(vec1, vec2),
+   rcode = (vec1 %*% vec2),
+   times=100))[, c(1, 4, 5)]  ##  end microbenchmark summary
> # Microbenchmark shows:
> # inner_vec() is several times faster than %*%, especially for lo
> #     expr     mean   median
> # 1 inner_vec 110.7067 110.4530
> # 2 rcode 585.5127 591.3575
```

Simulating *ARIMA* Processes Using *RcppArmadillo*

ARIMA processes can be simulated using *RcppArmadillo* even faster than by using the function `filter()`.

```
> # Source Rcpp functions from file
> Rcpp::sourceCpp(file="/Users/jerzy/Develop/lecture_slides/scripts/
> # Define AR(2) coefficients
> coeff <- c(0.9, 0.09)
> nrows <- 1e4
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> innov <- rnorm(nrows)
> # Simulate ARIMA using filter()
> arimar <- filter(x=innov, filter=coeff, method="recursive")
> # Simulate ARIMA using sim_ar()
> innov <- matrix(innov)
> coeff <- matrix(coeff)
> arimav <- sim_ar(coeff, innov)
> all.equal(drop(arimav), as.numeric(arimar))
> # Microbenchmark \emph{RcppArmadillo} code
> summary(microbenchmark(
+   rcpp = sim_ar(coeff, innov),
+   filter = filter(x=innov, filter=coeff, method="recursive"),
+   times=100))[, c(1, 4, 5)]  ##  end microbenchmark summary
```

```
// Rcpp header with information for C++ compiler
#include <RcppArmadillo.h> // include C++ header file fr
using namespace arma; // use C++ namespace from Armadill
// declare dependency on RcppArmadillo
// [[Rcpp::depends(RcppArmadillo)]]

//' @export
// [[Rcpp::export]]
arma::vec sim_ar(const arma::vec& innov, const arma::vec
  uword nrows = innov.n_elem;
  uword lookb = coeff.n_elem;
  arma::vec arimav[nrows);

  // startup period
  arimav(0) = innov(0);
  arimav(1) = innov(1) + coeff(lookb-1) * arimav(0);
  for (uword it = 2; it < lookb-1; it++) {
    arimav(it) = innov(it) + arma::dot(coeff.subvec(lookb
  }  // end for

  // remaining periods
  for (uword it = lookb; it < nrows; it++) {
    arimav(it) = innov(it) + arma::dot(coeff, arimav.sub
  }  // end for

  return arimav;
}  // end sim_arima
```

Fast Matrix Algebra Using *RcppArmadillo*

RcppArmadillo functions can be made even faster by operating on pointers to matrices and performing calculations in place, without copying large matrices.

RcppArmadillo functions can be compiled using the same *Rtools* as those for *Rcpp* functions:
https://cran.r-project.org/bin/windows/Rtools/

```
> library(RcppArmadillo)
> # Source Rcpp functions from file
> Rcpp::sourceCpp(file="/Users/jerzy/Develop/lecture_slides/scripts/
> matv <- matrix(runif(1e5), nc=1e3)
> # Center matrix columns using apply()
> matd <- apply(matv, 2, function(x) (x-mean(x)))
> # Center matrix columns in place using Rcpp demeanr()
> demeanr(matv)
> all.equal(matd, matv)
> # Microbenchmark \emph{RcppArmadillo} code
> library(microbenchmark)
> summary(microbenchmark(
+   rcode = (apply(matv, 2, mean)),
+   rcpp = demeanr(matv),
+   times=100))[, c(1, 4, 5)]  ##  end microbenchmark summary
> # Perform matrix inversion
> # Create random positive semi-definite matrix
> matv <- matrix(runif(25), nc=5)
> matv <- t(matv) %*% matv
> # Invert the matrix
> matrixinv <- solve(matv)
> inv_mat(matv)
> all.equal(matrixinv, matv)
> # Microbenchmark \emph{RcppArmadillo} code
> summary(microbenchmark(
+   rcode = solve(matv),
+   rcpp = inv_mat(matv),
+   times=100))[, c(1, 4, 5)]  ##  end microbenchmark summary
```

```
// Rcpp header with information for C++ compiler
#include <RcppArmadillo.h> // include C++ header file fr
using namespace arma; // use C++ namespace from Armadill
// declare dependency on RcppArmadillo
// [[Rcpp::depends(RcppArmadillo)]]

// Examples of \emph{RcppArmadillo} functions below

// The function demeanr() calculates a matrix with cente
// It accepts a pointer to a matrix and operates on the
// It returns the number of columns of the input matrix.
// It uses \emph{RcppArmadillo}.
//' @export
// [[Rcpp::export]]
int demeanr(arma::mat& matv) {
  for (uword i = 0; i < matv.n_cols; i++) {
    matv.col(i) -= arma::mean(matv.col(i));
  }  // end for
  return matv.n_cols;
}  // end demeanr

// The function inv_mat() calculates the inverse of symm
// definite matrix.
// It accepts a pointer to a matrix and operates on the
// It returns the number of columns of the input matrix.
// It uses \emph{RcppArmadillo}.
//' @export
// [[Rcpp::export]]
double inv_mat(arma::mat& matv) {
  matv = arma::inv_sympd(matv);
  return matv.n_cols;
}  // end inv_mat
```

Fast Correlation Matrix Inverse Using *RcppArmadillo*

RcppArmadillo can be used to quickly calculate the reduced inverse of correlation matrices.

```
> library(RcppArmadillo)
> # Source Rcpp functions from file
> Rcpp::sourceCpp("/Users/jerzy/Develop/lecture_slides/scripts/HighI
> # Calculate matrix of random returns
> matv <- matrix(rnorm(300), nc=5)
> # Reduced inverse of correlation matrix
> dimax <- 4
> cormat <- cor(matv)
> eigend <- eigen(cormat)
> invmat <- eigend$vectors[, 1:dimax] %*%
+   (t(eigend$vectors[, 1:dimax]) / eigend$values[1:dimax])
> # Reduced inverse using \emph{RcppArmadillo}
> invarma <- calc_inv(cormat, dimax=dimax)
> all.equal(invmat, invarma)
> # Microbenchmark \emph{RcppArmadillo} code
> library(microbenchmark)
> summary(microbenchmark(
+   rcode = {eigend <- eigen(cormat)
+ eigend$vectors[, 1:dimax] %*% (t(eigend$vectors[, 1:dimax]) / eige
+   rcpp = calc_inv(cormat, dimax=dimax),
+   times=100))[, c(1, 4, 5)]  ##  end microbenchmark summary
```

```
// Rcpp header with information for C++ compiler
// [[Rcpp::depends(RcppArmadillo)]]
#include <RcppArmadillo.h>
// include Rcpp C++ header files
using namespace stdev;
using namespace Rcpp; // use Rcpp C++ namespace
using namespace arma;

//' @export
// [[Rcpp::export]]
arma::mat calc_inv(const arma::mat& matv,
                   arma::uword dimax = 0, // Max number
                   double eigen_thresh = 0.01) { // Thre

  // Allocate SVD variables
  arma::vec svdval;  // Singular values
  arma::mat svdu, svdv;  // Singular matrices
  // Calculate the SVD
  arma::svd(svdu, svdval, svdv, tseries);
  // Calculate the number of non-small singular values
  arma::uword svdnum = arma::sum(svdval > eigen_thresh*a

  // If no regularization then set dimax to (svdnum - 1)
  if (dimax == 0) {
    // Set dimax
    dimax = svdnum - 1;
  } else {
    // Adjust dimax
    dimax = stdev::min(dimax - 1, svdnum - 1);
  }  // end if

  // Remove all small singular values
  svdval = svdval.subvec(0, dimax);
  svdu = svdu.cols(0, dimax);
  svdv = svdv.cols(0, dimax);

  // Calculate the reduced inverse from the SVD decompos
  return svdv*arma::diagmat(1/svdval)*svdu.t();
```

Portfolio Optimization Using *RcppArmadillo*

Fast portfolio optimization using matrix algebra can be implemented using *RcppArmadillo*.

```
// Fast portfolio optimization using matrix algebra and \emph{RcppArmadillo}
arma::vec calc_weights(const arma::mat& returns, // Asset returns
                       Rcpp::List controll) { // List of portfolio optimization parameters

  // Apply different calculation methods for weights
  switch(calc_method(method)) {
  case methodenum::maxsharpe: {
    // Mean returns of columns
    arma::vec colmeans = arma::trans(arma::mean(returns, 0));
    // Shrink colmeans to the mean of returns
    colmeans = ((1-alpha)*colmeans + alpha*arma::mean(colmeans));
    // Calculate weights using reduced inverse
    weights = calc_inv(covmat, dimax, eigen_thresh)*colmeans;
    break;
  }  // end maxsharpe
  case methodenum::maxsharpemed: {
    // Median returns of columns
    arma::vec colmeans = arma::trans(arma::median(returns, 0));
    // Shrink colmeans to the median of returns
    colmeans = ((1-alpha)*colmeans + alpha*arma::median(colmeans));
    // Calculate weights using reduced inverse
    weights = calc_inv(covmat, dimax, eigen_thresh)*colmeans;
    break;
  }  // end maxsharpemed
  case methodenum::minvarlin: {
    // Minimum variance weights under linear constraint
    // Multiply reduced inverse times unit vector
    weights = calc_inv(covmat, dimax, eigen_thresh)*arma::ones(ncols);
    break;
  }  // end minvarlin
  case methodenum::minvarquad: {
    // Minimum variance weights under quadratic constraint
    // Calculate highest order principal component
    arma::vec eigenval;
    arma::mat eigenvec;
```

Strategy Backtesting Using *RcppArmadillo*

Fast backtesting of strategies can be implemented using *RcppArmadillo*.

```
arma::mat roll_portf(const arma::mat& excess, // Asset excess returns
                     const arma::mat& returns, // Asset returns
                     Rcpp::List controll, // List of portfolio optimization model parameters
                     arma::uvec startp, // Start points
                     arma::uvec endd, // End points
                     double lambdaf = 0.0, // Decay factor for averaging the portfolio weights
                     double coeff = 1.0, // Multiplier of strategy returns
                     double bidask = 0.0) { // The bid-ask spread

  double lambda1 = 1-lambdaf;
  arma::uword nweights = returns.n_cols;
  arma::vec weights(nweights, fill::zeros);
  arma::vec weights_past = ones(nweights)/stdev::sqrt(nweights);
  arma::mat pnls = zeros(returns.n_rows, 1);

  // Perform loop over the end points
  for (arma::uword it = 1; it < endd.size(); it++) {
    // cout << "it: " << it << endl;
    // Calculate the portfolio weights
    weights = coeff*calc_weights(excess.rows(startp(it-1), endd(it-1)), controll);
    // Calculate the weights as the weighted sum with past weights
    weights = lambda1*weights + lambdaf*weights_past;
    // Calculate out-of-sample returns
    pnls.rows(endd(it-1)+1, endd(it)) = returns.rows(endd(it-1)+1, endd(it))*weights;
    // Add transaction costs
    pnls.row(endd(it-1)+1) -= bidask*sum(abs(weightv - weights_past))/2;
    // Copy the weights
    weights_past = weights;
  }  // end for

  // Return the strategy pnls
  return pnls;

}  // end roll_portf
```

Package *reticulate* for Running Python from RStudio

The package *reticulate* allows running Python functions and scripts from RStudio.

The package *reticulate* relies on Python for interpreting the Python code.

```
> # Install package reticulate
> install.packages("reticulate")
> # Start Python session
> reticulate::repl_python()
> # Exit Python session
> exit
```

You must set your Global Options in RStudio to your Python executable, for example:
/Library/Frameworks/Python.framework/Versions/3.10/bin/python3.10

You can learn more about the package *reticulate* here:
https://rstudio.github.io/reticulate/

Running Python Under *reticulate*

```
"""
Script for loading OHLC data from a CSV file and plotting a candlestick plot.
"""
# Import packages
import pandas as pd
import numpy as np
import plotly.graph_objects as go
# Load OHLC data from csv file - the time index is formatted inside read_csv()
symbol = "SPY"
range = "day"
filename = "/Users/jerzy/Develop/data/" + symbol + "_" + range + ".csv"
ohlc = pd.read_csv(filename)
datev = ohlc.Date
# Calculate log stock prices
ohlc[["Open", "High", "Low", "Close"]] = np.log(ohlc[["Open", "High", "Low", "Close"]])
# Calculate moving average
lookback = 55
closep = ohlc.Close
pricema = closep.ewm(span=lookback, adjust=False).mean()
# Plotly simple candlestick with moving average
# Create empty graph object
plotfig = go.Figure()
# Add trace for candlesticks
plotfig = plotfig.add_trace(go.Candlestick(x=datev,
  open=ohlc.Open, high=ohlc.High, low=ohlc.Low, close=ohlc.Close,
  name=symbol+" Log OHLC Prices", showlegend=False))
# Add trace for moving average
plotfig = plotfig.add_trace(go.Scatter(x=datev, y=pricema,
  name="Moving Average", line=dict(color="blue")))
# Customize plot
plotfig = plotfig.update_layout(title=symbol + " Log OHLC Prices",
  title_font_size=24, title_font_color="blue", yaxis_title="Price",
  font_color="black", font_size=18, xaxis_rangeslider_visible=False)
# Customize legend
plotfig = plotfig.update_layout(legend=dict(x=0.2, y=0.9, traceorder="normal",
  itemsizing="constant", font=dict(family="sans-serif", size=18, color="blue")))
# Render the plot
plotfig.show()
```

Homework Assignment

No homework!